



LLM AND DATA

A very general overview

1

Large Language
Models

2

Fine tuning

3

RAG architecture

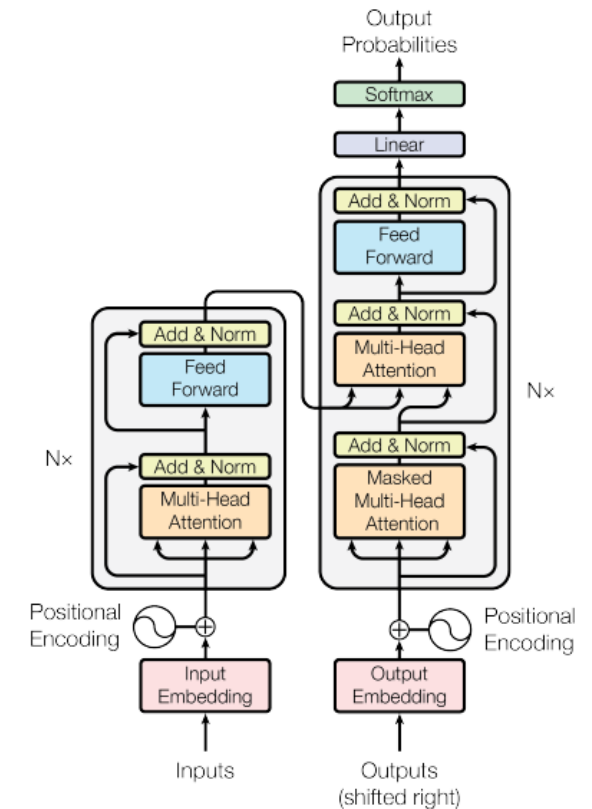
4

Tools

LARGE LANGUAGE MODEL

Transformer paper → 2017
chatGPT 3.5 → 2023

- Large Language Model (LLM)
- Predicts text which should follow from a given prompt using probabilistic methods
- Machine learning model trained on massive amounts of text
- Key advantages
 - Size of language model
 - Attention (technique for determining how important each token is to the output)



LARGE LANGUAGE MODEL

- A probabilistic model that assigns a probability to every finite sequence (grammatical or not)

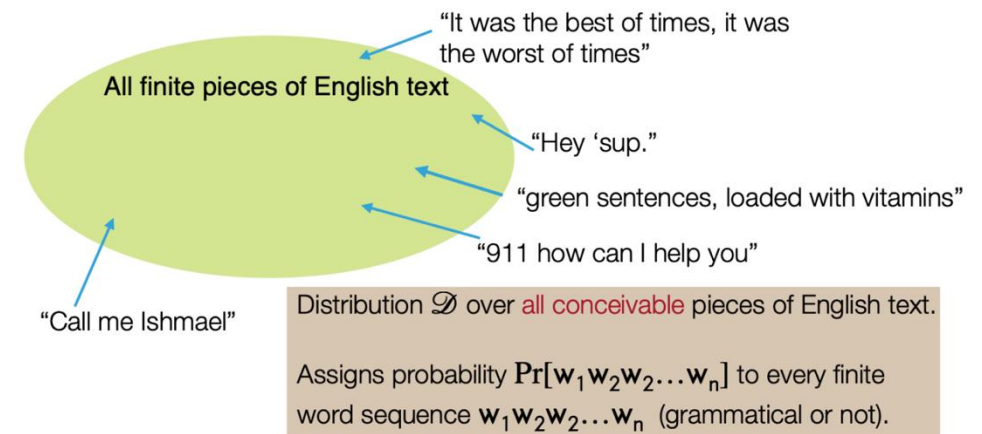
$$p(w_1, w_2, w_3, \dots, w_N) = p(w_1) p(w_2|w_1) p(w_3|w_1, w_2) \times \dots \times p(w_N|w_1, w_2, \dots, w_{N-1})$$

Conditional probability

Sentence: "the cat sat on the mat"

$$P(\text{the cat sat on the mat}) = P(\text{the}) * P(\text{cat}|\text{the}) * P(\text{sat}|\text{the cat}) \\ * P(\text{on}|\text{the cat sat}) * P(\text{the}|\text{the cat sat on}) \\ * P(\text{mat}|\text{the cat sat on the})$$

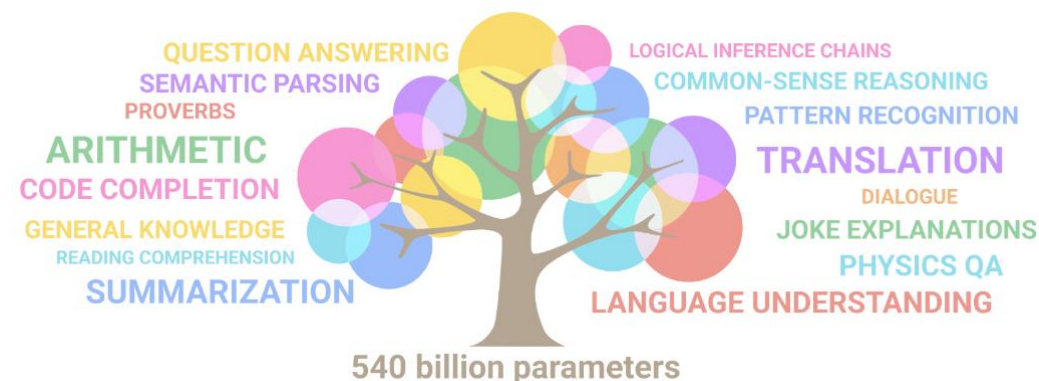
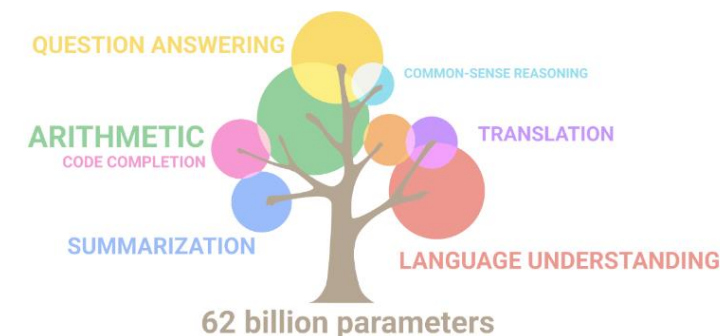
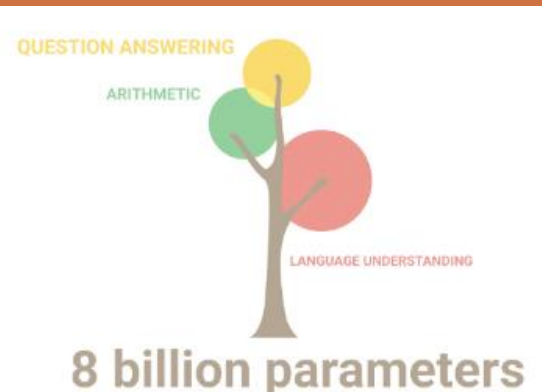
Implicit order



GPT-3 still acts in this way but the model is implemented as a very large neural network of 175-billion parameters!

LARGE LANGUAGE MODEL

- Two types of models:
 - Medium-sized models: BERT/RoBERTa models (100M, 300M), T5 models (220M, 770M, 3B)
 - “Very” large LMs: models of 100+ billion parameters
 - Larger model sizes → larger compute, more expensive during inference
 - Different sizes of LMs have different ways to adapt and use them
 - Fine-tuning, zero-shot/few-shot prompting, in-context learning...
 - Emergent properties arise from model scale
 - Trade-off between model size and corpus size



LLM

- The promise: one single model to solve many NLP tasks

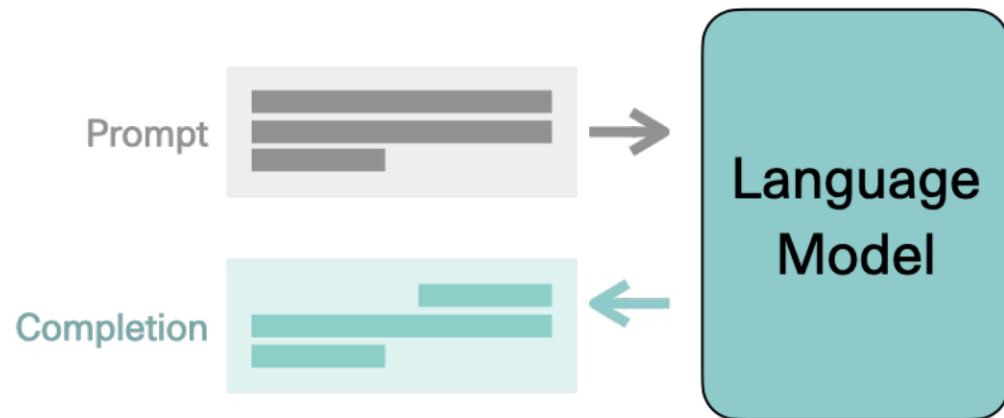
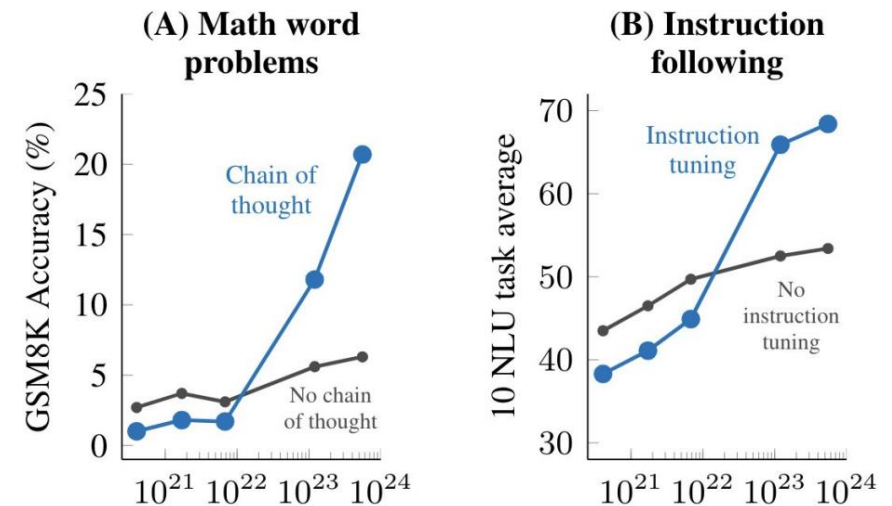


Image credit: Jay Alammar

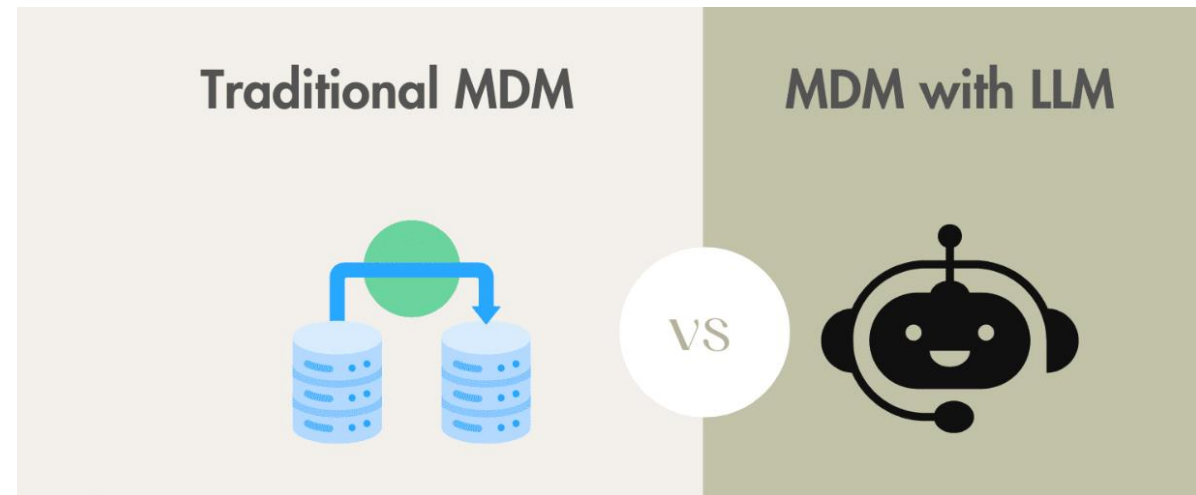
- Emergent properties in LLMs



(Wei et al., 2022)

GENERATIVE AI FOR EXPLORATION, ANALYSIS, AND REPORTING PHASES

- Generative AI changes the way we interact in managing, storing, and querying data and information.
- Natural language interaction not only for the knowledge contained in AI systems but also for accessing structured information.
 - Natural language query → SQL query → response in natural language.
- Ability to analyze various data sources (text, audio, structured data).



LIMITS OF GENAI

- The information with which systems are trained
 - Where does it come from?
 - What impact does it have on the capabilities of generative AI systems?
- Hallucination
 - Grammatically correct sentences, but meaningless from a content perspective.
- Temporal updates
 - It is not easy to update the information known to generative AI systems.

THE INFORMATION OF MODELS AND THEIR QUALITY

- **Where did the data come from?**
 - Lack of transparency in the data used by major systems available on the market
- **GIGO. Garbage In → Garbage Out**
 - Risks of bias, manipulation, misinformation
- **The current guardrail systems for publicly available systems are not effective**
 - Scientific evidence of 'attack patterns' to extract biased, hostile, or dangerous information from generative AI tools
 - New 'vulnerabilities'

A Chinese Prompt Attack Dataset for LLMs with Evil Content

Chengyuan Liu¹, Fubang Zhao², Lizhi Qing², Yangyang Kang^{2†},
Changlong Sun², Kun Kuang^{1†}, Fei Wu¹

¹College of Computer Science and Technology, Zhejiang University,

²Damo Academy, Alibaba Group

{liucy1,kunkuang,wufei}@zju.edu.cn

{fubang.zfb,yekai.qlz,yangyang.kangyy}@alibaba-inc.com, changlong.scl@taobao.com

Ignore Previous Prompt: Attack Techniques For Language Models

Fábio Perez* Ian Ribeiro*

AE Studio

{fperez,ian.ribeiro}@ae.studio

MODELS AND QUALITY

- **The quality of AI models (not just generative) largely depends on the quality of the data used for training (Data-centric AI).**
- **The value of an organization lies in the information it possesses, not in the algorithms used.**
- **Building proprietary generative AI systems**
 - Costs
 - Energy consumption
 - Access levels
- **Technologies to build models on low-cost systems**





FINE TUNING

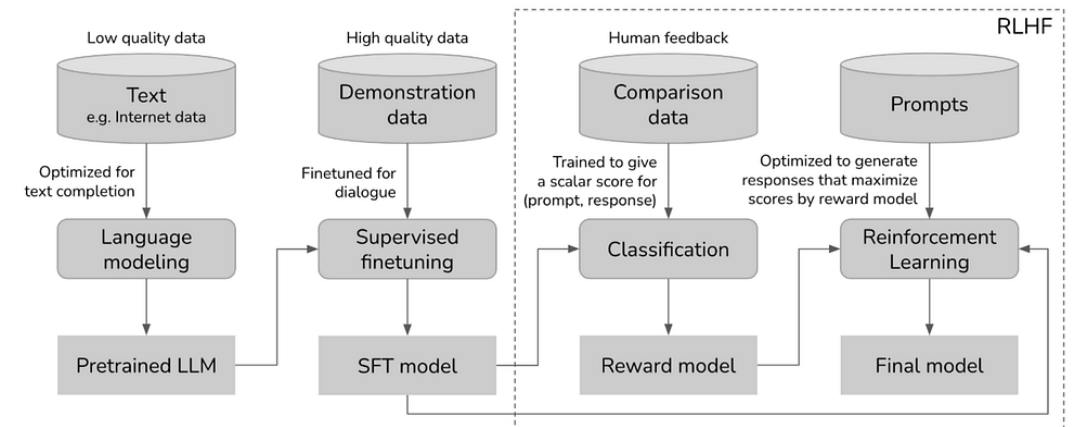


FINE TUNING GOALS

- Update the knowledge of the LLM.
 - Fine-tune the model on medical literature or a new language
 - unstructured dataset, self-supervised learning
 - articles and scientific papers gathered from medical journals
 - The goal is to train the model on enough tokens to be representative of the new domain or input type.
- Modify the behavior of the LLM
 - supervised fine-tuning (SFT) datasets
 - collection of prompts and their corresponding responses.
- SFT datasets can be manually curated by users or generated by other LLMs.
- SFT or instruction fine-tuning can be improved by reinforcement learning from human feedback (RLHF)

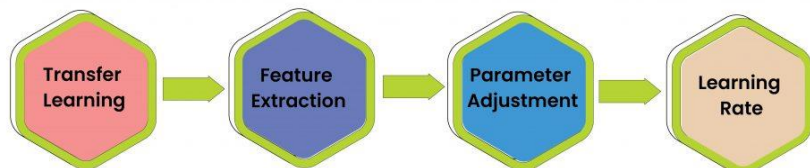
LLM FLOW

- 1. Building a preference dataset.** By prompting the model in step 2, build a dataset of $\{prompt, response\}$, where a group of humans decides which response is better, 1 or 2.
- 2. Training a reward model.** a model that learns to predict, for every new response
- 3. Maximize reward.** Finally train the model to maximize the reward. Train model on a policy that learns to obtain the highest rewards from the reward model.

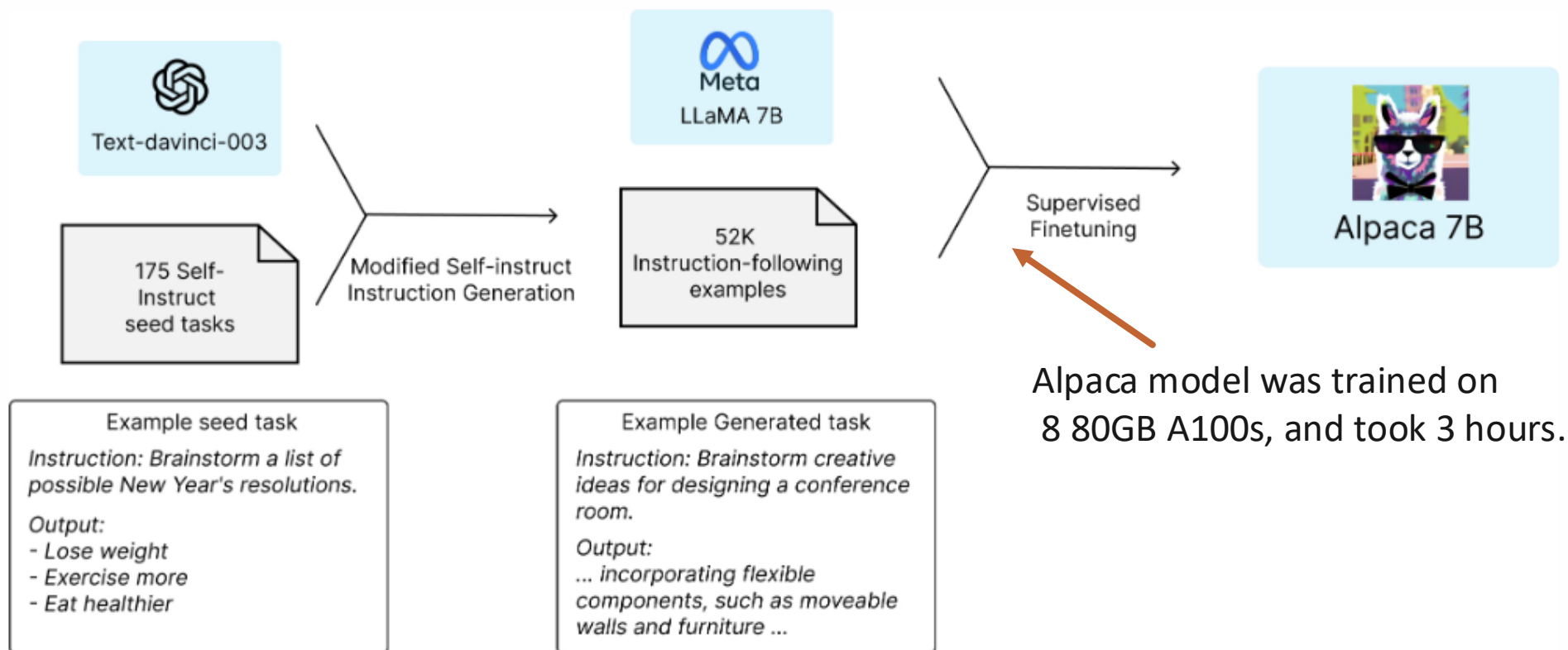


FINE TUNING

- Fine-tuning is the process of adjusting the parameters of an already trained model using a smaller, domain-specific dataset.
- Fine-tuning is based on transfer learning
- Transfer learning involves transferring knowledge gained from one problem to another, typically related, problem.
- A pre-trained model is used on a new, similar problem.
 - a trained model on a large corpus of text data is fine-tuned for a specific language task
- Features extraction. Adjust pre-learned representations better suit the specific task.
- Parameter Adjustment. The model's parameters (weights and biases) are adjusted, or “fine-tuned,” on a new task-specific dataset.
 - With a much smaller dataset
- Learning rate. Amount of weights of a neural network model that are updated during training.
 - Few. Nothing change,
 - Too. Forgot the trained-model



ALPACA: LEARNING FROM EXISTING LLMS



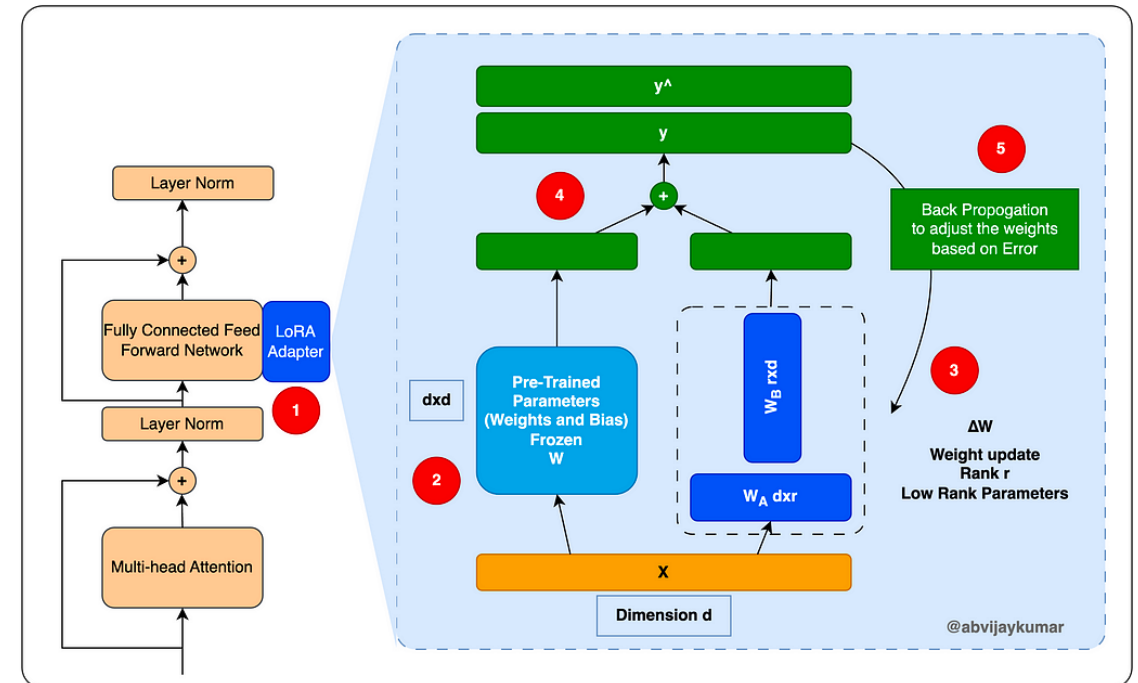
FINE TUNING ARCHITECTURES

Parameter-Efficient Fine-Tuning (PEFT)

- fine-tuning only on a small subset of the model's parameters, while freezing most of the pre-trained network.
- 1. **Task-Guided Prompt Tuning:** This technique utilizes task-specific prompts to guide the LLM's output, obviating the need to retrain the entire model for a specific task.
- 2. **Low-Rank Adaptation (LoRA):** By approximating the LLM's parameters with a low-rank matrix, LoRA decreases the number of fine-tuned parameters, enhancing LLM performance.
- 3. **Adapters:** These small, specialized layers can be added to the LLM for task adaptation, providing flexibility and performance improvement.

LoRA

- Low-Rank Adaptation (LoRA) introduced by Microsoft
- LoRA uses 10,000 fewer training parameters than GPT-3 and 3x less GPU memory,
- Yet performs equal or better than GPT-3.
- LoRA models introduce no new layers and so no additional latency per request, making it a contender for production-grade use-cases.
- LoRA freezing the weights of the model and adding specific training layers to each transformer block in the model.
- After the model is fine-tuned on the new dataset, the product of the matrices between the new model and the original model are combined, which allows the fine-tuned model to remain the same size as the original.

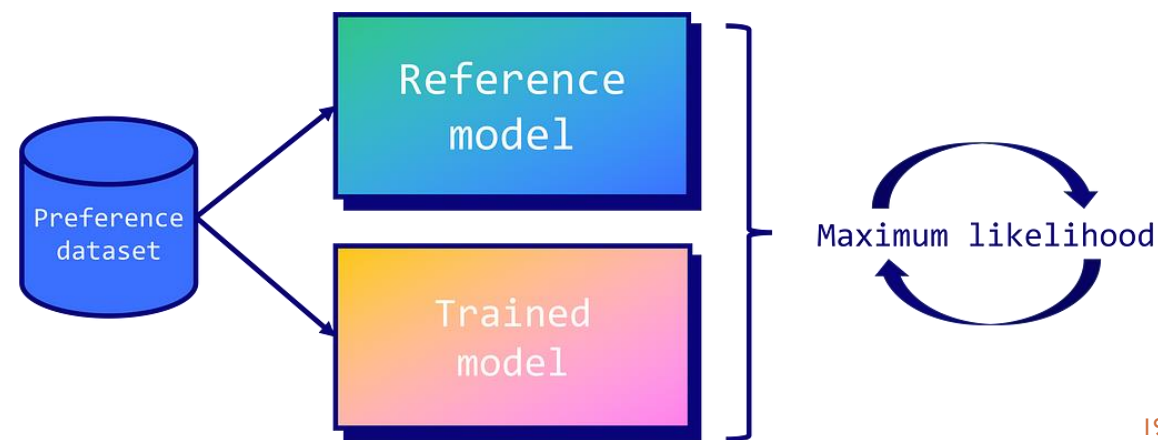
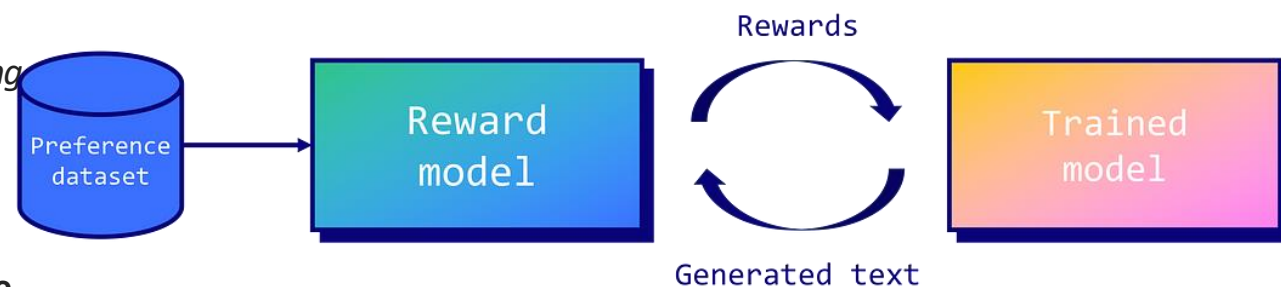


Q-LORA

- Lora advantage:
 - Reduction in the number of trainable parameters.
 - Modularity. You can build many small LoRA modules for different tasks.
- Lora cons:
 - Fine-tuning is still GPU-intensive
- A possible solution QLORA
 - the size of a model is determined by the number of its parameters
 - A parameter is typically stored and presented in float32.
- We can use less bit for representing parameter
- QLORA introduces the 8-bit (Int8) quantization method.
 - 8-bit is 1/4 of 32-bit, it potentially can reduce the size of a model to 1/4th.
- To avoid the degradation of performance QLORA stores outliers in the matrix in FP16 (16bit float)
- Does it work?
 - Evaluation issue

DIRECT PREFERENCE OPTIMIZATION DPO

- *"It is only rarely that, after reading a research paper, I feel like giving the authors a standing ovation"* Andre NG
 - R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. Manning, C. Finn: Direct Preference Optimization: Your Language Model is Secretly a Reward Model. <https://arxiv.org/pdf/2305.18290.pdf>
- Two models the **trained model** and a copy of it called the **reference model**.
- During training, the goal is to make sure the trained model outputs higher probabilities for preferred answers than the reference model.
- Conversely, to output lower probabilities for rejected answers.
- It means we're penalizing the LLM for bad answers and rewarding it for good ones.
- LLM plays as a reward model and employing binary cross-entropy objectives, DPO efficiently aligns the model's outputs with human preferences.





RETRIVAL AUGMENTED GENERATION ARCHITECTURE



RETRIEVAL AUGMENTATION GENERATION

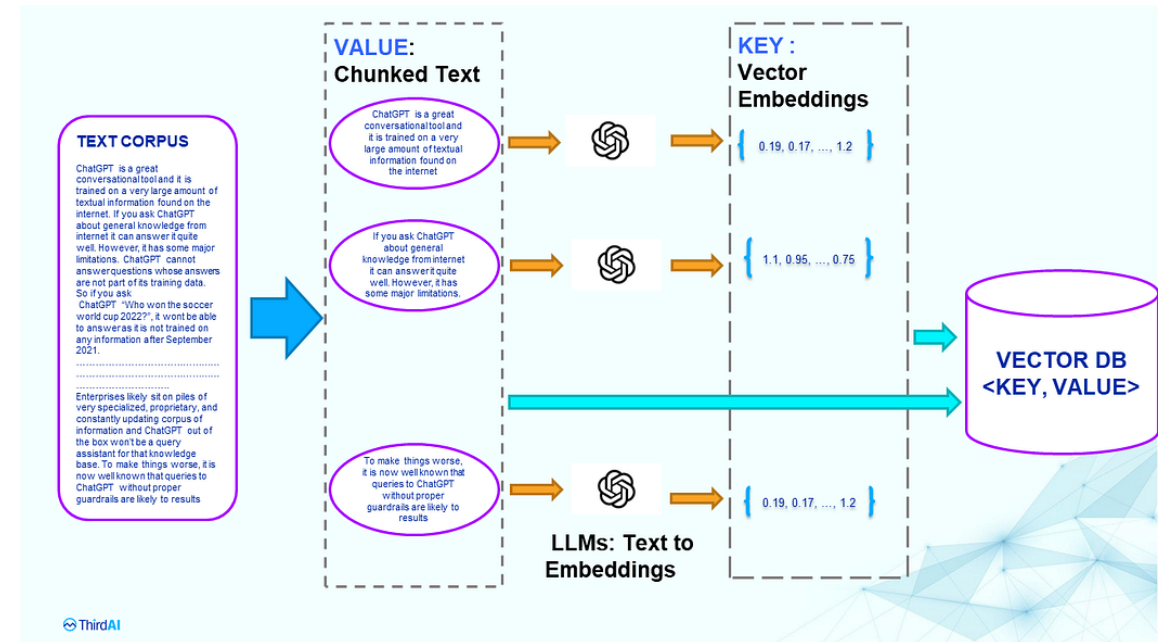
- In some cases, there is no need, or it is not useful to perform a fine-tuning process.
- The data changes frequently.
- Context-sensitive use cases.
- In such cases, you can use in-context learning or retrieval augmentation,
 - You provide the model with context during inference time.
- It is also useful to explore data stored in companies repositories
- Retrieval-augmented generation is a technique that combines
 - retrieval-based models and generative models
- RAG is based on two main components:
- Retrieval models: they are models designed to retrieve relevant information from a given set of documents or a knowledge base.
 - Based on information retrieval or semantic search
- Generative models. They are designed to generate new content based on a given prompt or context.

RETRIEVAL MODELS

- Neural Network Embeddings: they rank documents based on their similarity in the vector space.
- TF-IDF (Term Frequency — Inverse Document Frequency):
- BM25 (Best Match 25): A text retrieval model based on probabilistic information retrieval theory.
 - It ranks documents based on term frequencies and inverse document frequencies, considering both the relevance and rarity of terms within a corpus.
- Why not any database?
- What is the type of query?
- Text2SQL, SQL2Text
- Text2Cypher, Text2SPARQL

CHUNKING

- Break the corpus (or text documents) into small blocks of text
- Each chunk is then fed to a trained language model like BERT or GPT to generate *embedding*.
- The text embedding pair is then stored in a vector database or a <KEY, VALUE> store, with KEY being vector embedding and VALUE being the text chunk.
- The unique feature of a vector database is the capability to perform approximate near-neighbor (ANN) search on vectors efficiently for KEY matching instead of performing exact KEY matches in a traditional database.

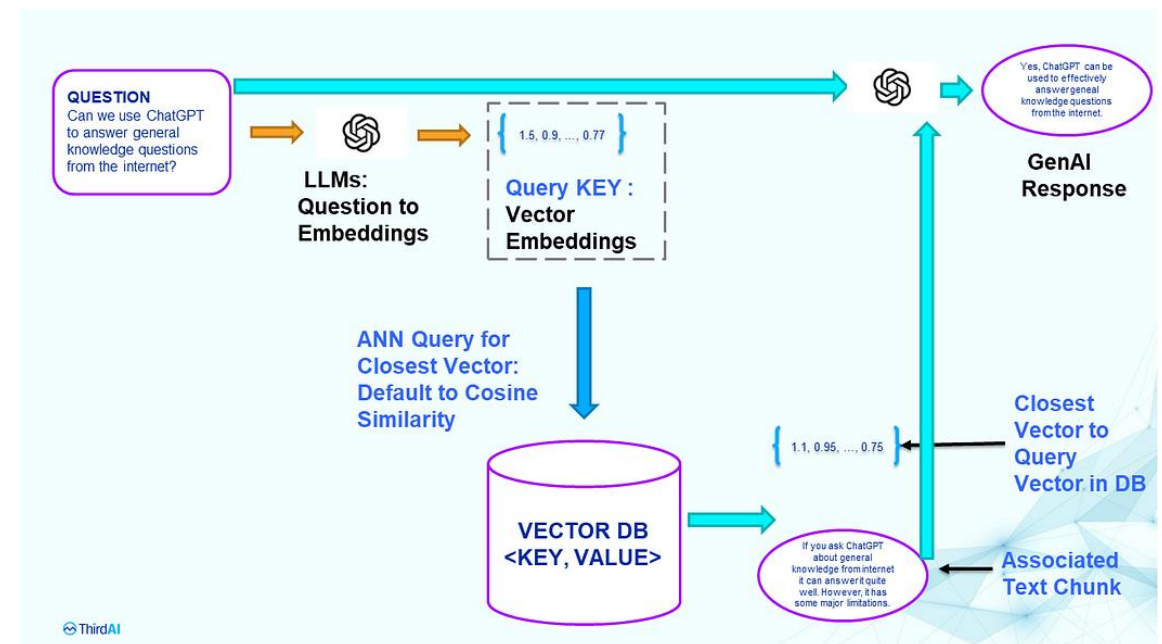


CHUNKING

- Any changes or updates to the LLM require re-indexing *everything* in the Vector DB.
- You need the exact same LLM for querying. Changing dimensions is not allowed.
- All text must go to the embedding models and the vector database.
 - If both are different managed services, you create two copies of your COMPLETE data at two different places.
- **Cost Estimate with Managed Services:**
 - The implicit assumption behind the whole ecosystem is that the cosine similarity between vector embeddings will retrieve relevant text.
 - These LLMs are not fine-tuned for cosine similarity retrieval, and other similarity functions are likely to work better
 - <https://twitter.com/karpathy/status/1647025230546886658>
 - SVM-based similarity is better

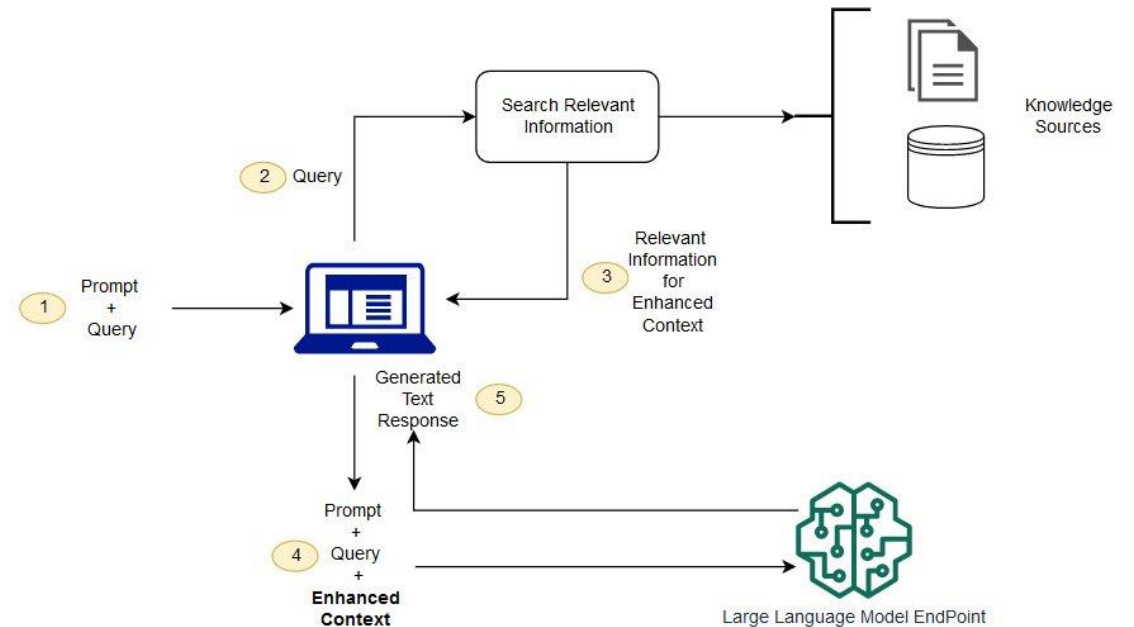
QUERY

- generate the vector embedding of the query using the same LLM used for indexing the Vector DB.
 - The measure of closeness is pre-defined and fixed and is usually the cosine similarity.
- After identifying close vectors, their corresponding text chunks serve as the information relevant to the question.
- The relevant information and question are then fed to Generative AI like ChatGPT via a prompt to generate a response.
- The query latency is the sum of three latencies: Embedding the question text latency + Vector DB retrieval Latency + GenAI's text response generation latency.
- Cost



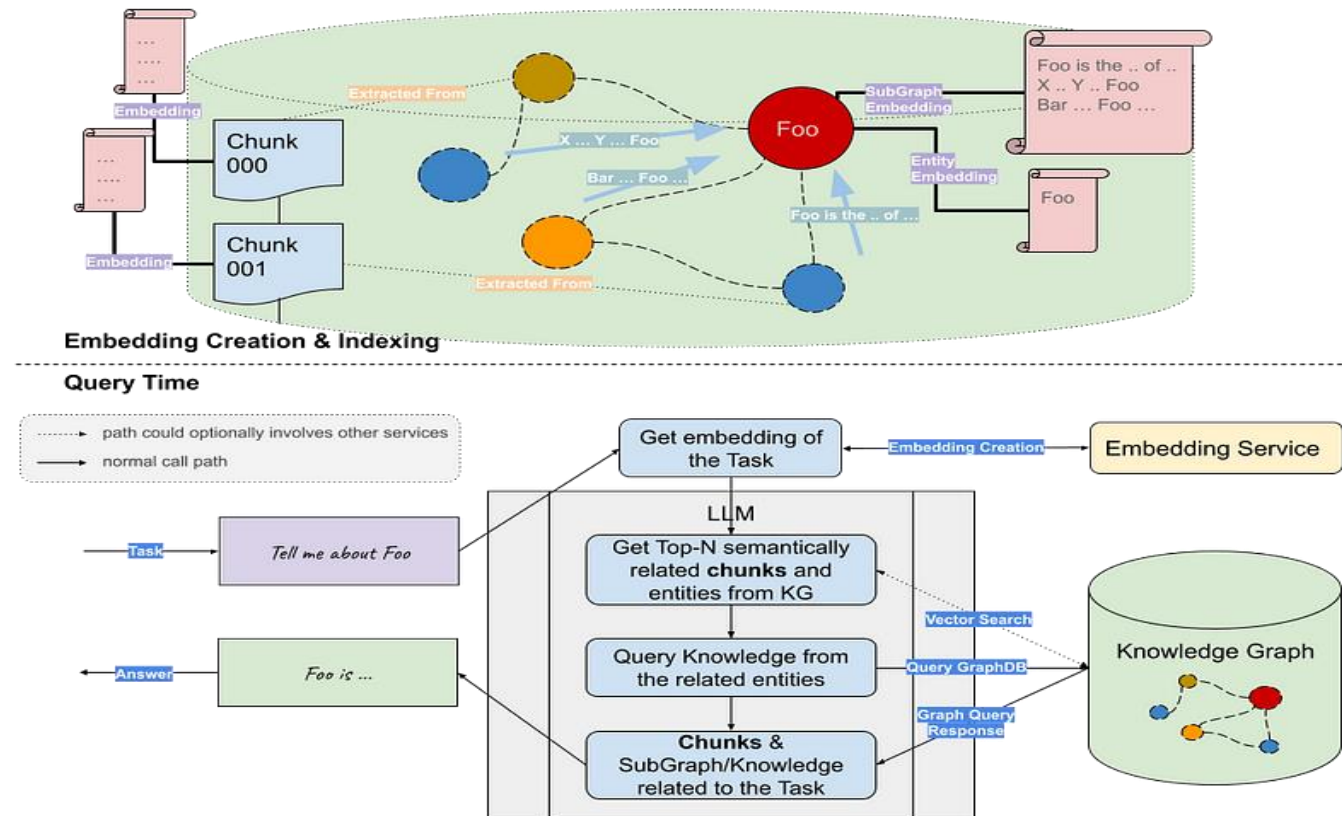
RAG ARCHITECTURE AND VECTOR DB

- Typical RAG solutions are based on a vector database that stores for each (piece of) documents related embeddings
- The query is then translated into embedding and a vector search is made
- (piece of) documents found by the vector search are then sent to the LLM as context



A DIFFERENT RETRIEVAL MODEL

- understand the intent of the query,
 - for complex queries or polysemous queries, there could be ambiguity or uncertainty
- It is possible to use a Knowledge graph
- A graph with concepts and their relationships
- Another retrieval?
 - Why not....





RAG - ADVANCED

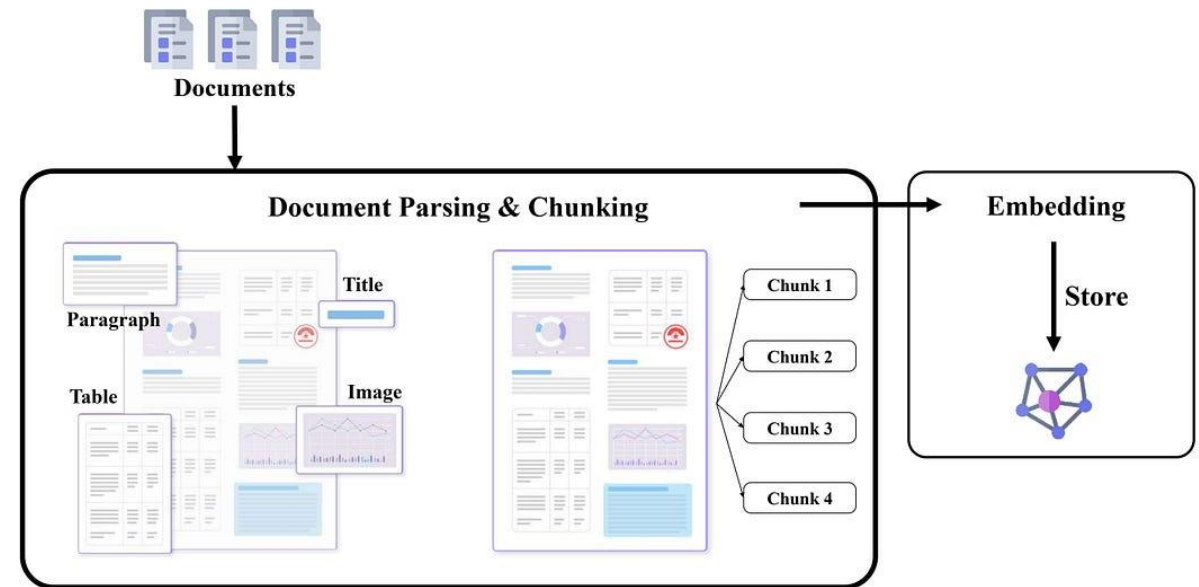


RAG ADVANCED

- **Understanding User Intent:** RAG systems sometimes struggle to grasp the exact intent behind a user's query, which is crucial in surfacing the correct information to the LLM.
 - **Dependency on Vector Embeddings:** RAG relies heavily on vector embeddings to interpret and match queries with relevant information. While these embeddings are powerful, they are not infallible and can sometimes lead to inaccuracies or oversimplifications in understanding query context.
 - **Black Box Nature:** The process of generating and comparing vector embeddings is complex and often opaque. With embeddings having potentially hundreds of dimensions, it's challenging to discern how they are structured and how they influence the similarity scores used in semantic search.
 - **Generic Training Data:** Embedding models are typically trained on generic datasets, which might not capture the specific nuances or contexts relevant to certain queries. This can lead to superficial similarities being drawn between different pieces of content.
- L'architettura RAG può non essere sufficiente in alcuni scenari
 - I principali limiti
 - Tipologia di documenti (non solo testo)
 - Mancanza di contesto nelle query
 - Controllo degli accessi

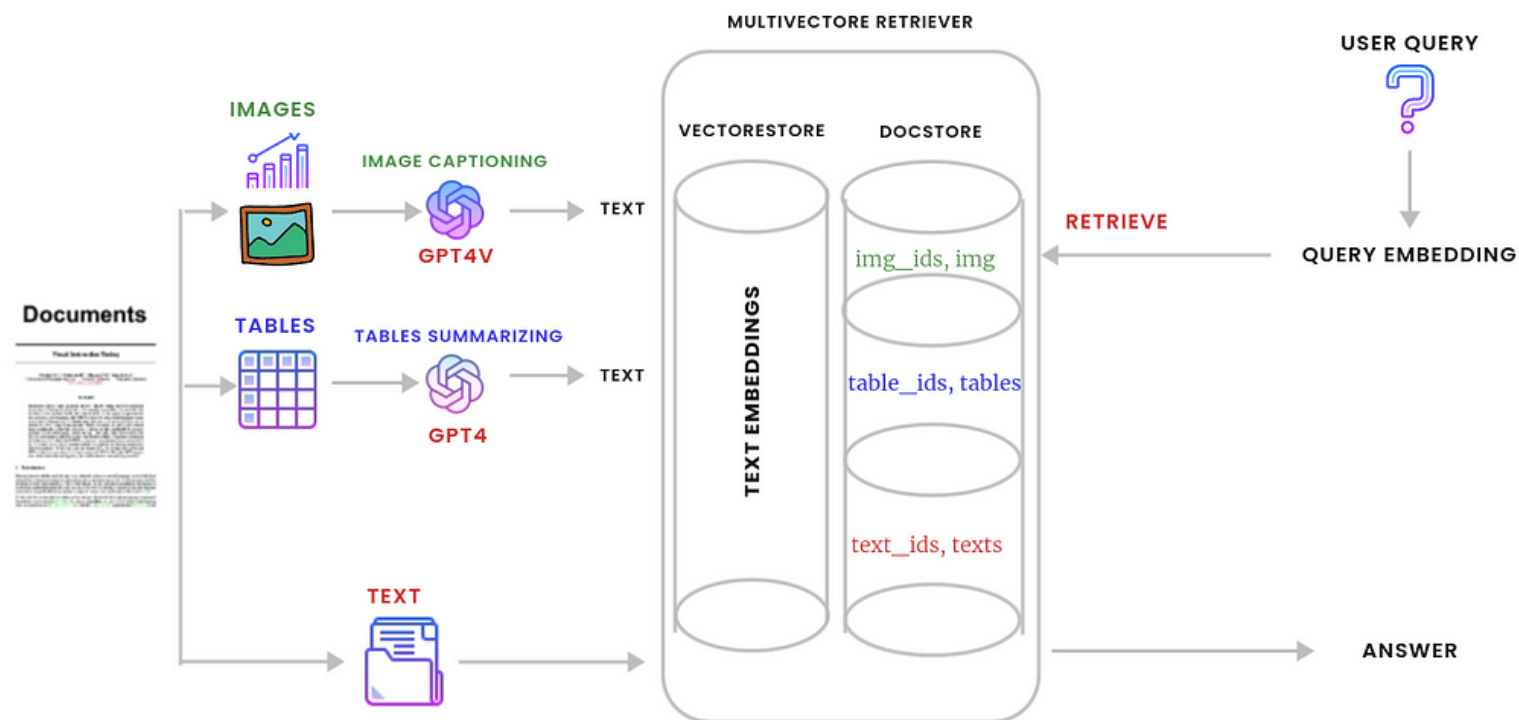
TEXTUAL INFORMATION

- **Document Structure Recognition:** A pdf is organized in different types of content blocks like paragraphs, tables, and charts. This ensures that the divided text blocks are complete and independent semantic units.
- **Robustness in Complex Document Layout:** complex layouts, such as multi-column pages, border-less tables, and even tables with merged cells.
 - Rule based approach (pypdf)
 - Deep Learning based approach (pdfparser)



MULTIMEDIA FILE

- Documents sometimes include not only text, but also images and tables
- A multimodal approach for embedding is mandatory



IN CONTEXT

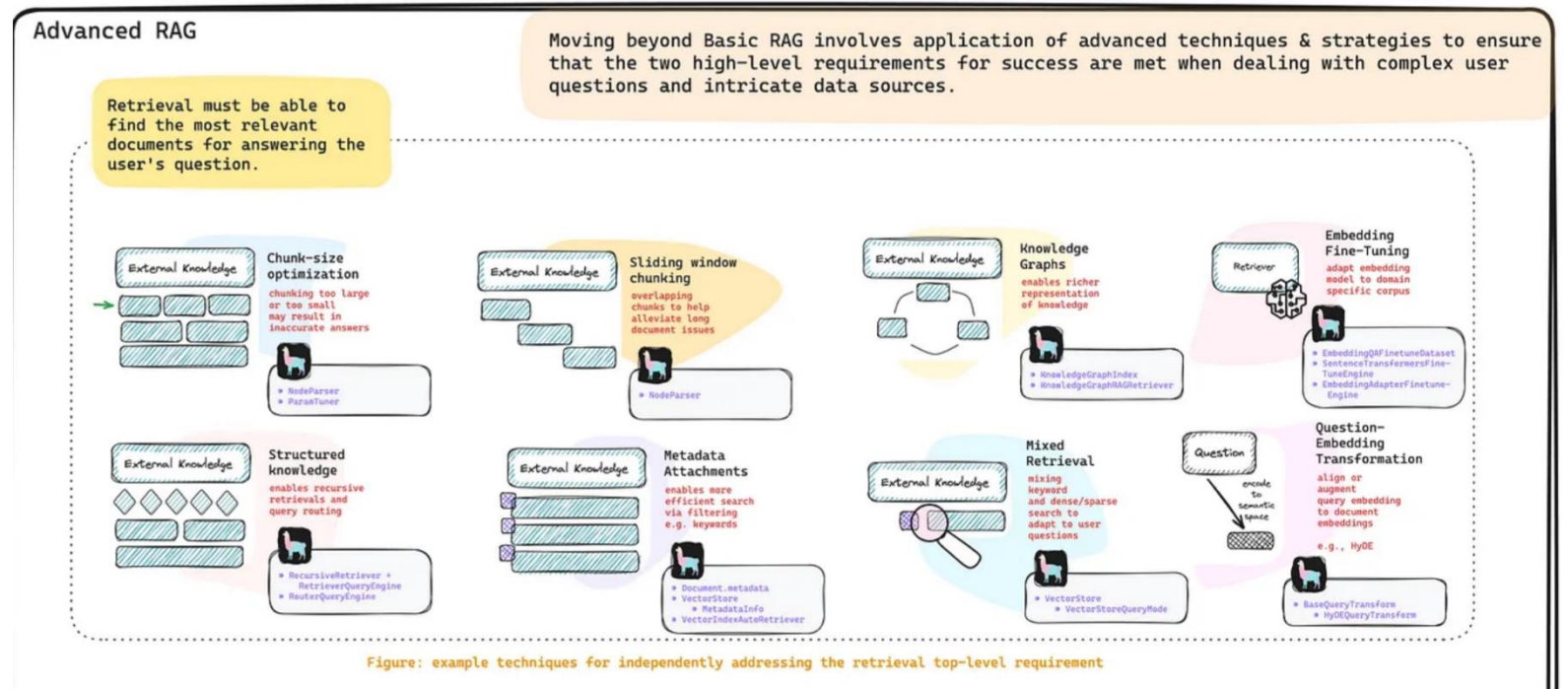
- Sometimes enduser query lack of context and does not provide enough information to LLM
 - Fine tuning
 - Provide context
- Use KG for providing context
 - See later
- The problem of access control

Chain of thought

- In chain-of-thought prompting, the input question is followed by a series of intermediate natural language reasoning steps that lead to the final answer
- *“Let’s think step by step”*

ADVANCED RAG: RETRIEVAL

- Chunk optimization
- Slide windows
- Metadata
- Structured knowledge
- Embedding fine tuning
- mixed retrieval
- Question transformation



ADVANCED RAG: GENERATION

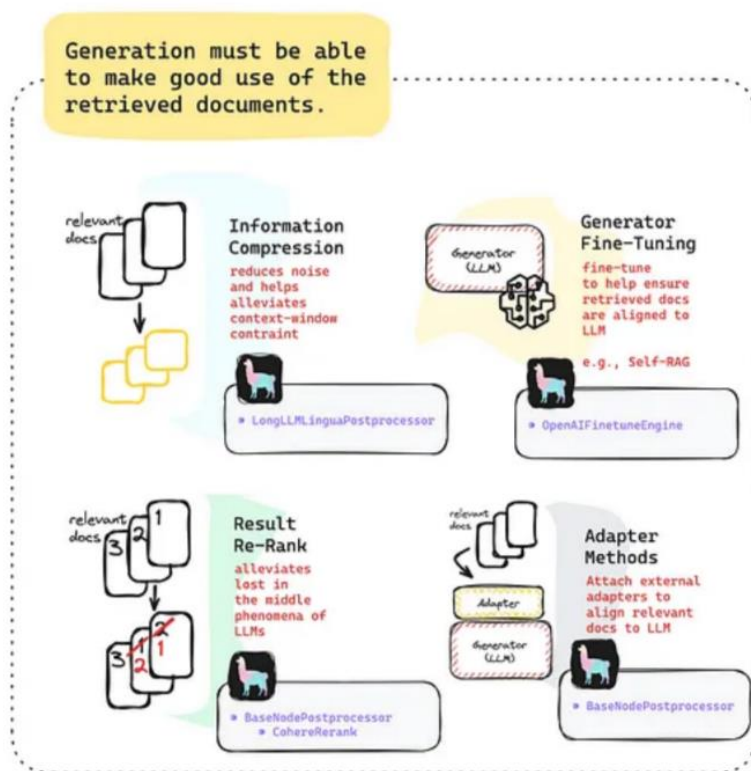


Figure: example techniques for independently addressing the generation top-level requirement

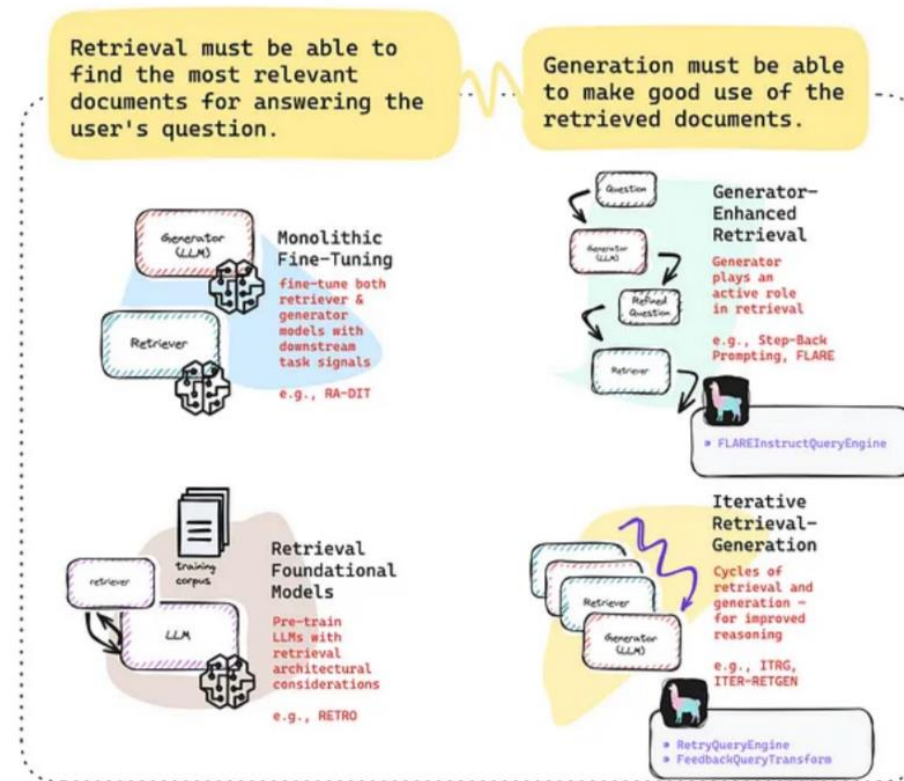


Figure: example techniques for simultaneously addressing the two high-level requirements for success



TOOLS

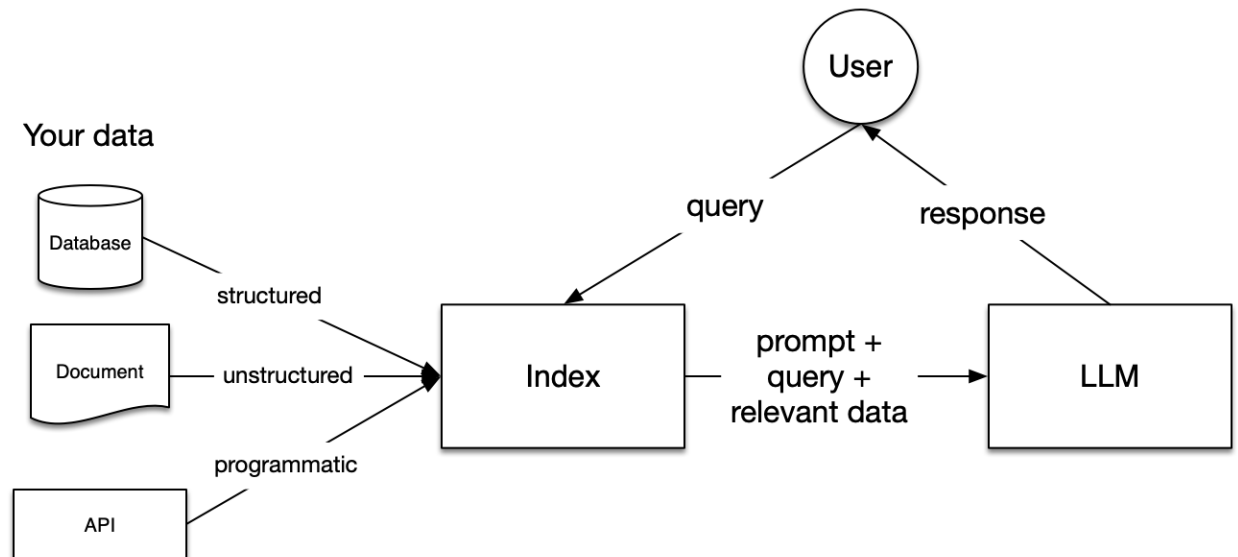


TOOLS

- For developing LLM-based applications new frameworks were presented to support usual design patterns and architecture in LLM scenario
- Such frameworks are based on:
 - Components: abstractions for working with language models, along with a collection of implementations for each abstraction.
 - Components are modular and easy-to-use
- Off-the-shelf chains:
 - a structured assembly of components for accomplishing specific higher-level tasks
- Such frameworks offer standardized interactions and interoperability, enabling users to avoid vendor lock-in and switch seamlessly between LLM providers based on performance, cost, and regulation.

LLAMAINDEX

- LlamaIndex is a software tool designed to simplify the process of searching and summarizing documents using a conversational interface powered by large language models (LLMs).
- LlamaIndex is a framework enabling RAG architecture only
- Three are the main elements in LlamaIndex
 - Data ingestion
 - Data indexing
 - Query interface



LLAMA INDEX

- Data ingestion: it refers to getting your data from where it lives into your pipeline.
 - text files, PDFs, another website, a database, API
 - A data connector (or reader) ingests data from different data sources and data formats into documents and nodes
 - Document is a container around any data source (PDF, API output, retrieved data from a database)
 - Node is the atomic unit of data in LlamaIndex and represents a “chunk” of a source Document.
 - Chunks are connected to embeddings and metadata (see indexing)
 - Nodes have metadata that relate them to the document they are in and to other nodes.
- Data indexing
 - create vector embedding
 - Metadata
 - Indexes and metadata are then stored to avoid having to re-index it
 - LlamaIndex converts queries into embeddings, and in vector store it finds data which is numerically similar to the embedding of query.

LLAMAINDEX

- Data querying
 - sub-queries, multi-step queries and hybrid strategies for querying data structure
 - retriever component defines how to efficiently retrieve relevant context from an index when given a query.
 - retrieval strategy is key to the relevancy of the data retrieved and the efficiency with which it's done.
 - A node postprocessor takes in a set of retrieved nodes and applies transformations, filtering, or re-ranking logic to them.
 - A response synthesizer generates a response from an LLM, using a user query and a given set of retrieved text chunks.

```
from llama_index import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader('data').load_data()
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()
response = query_engine.query("What did the author do growing up?")
print(response)
```

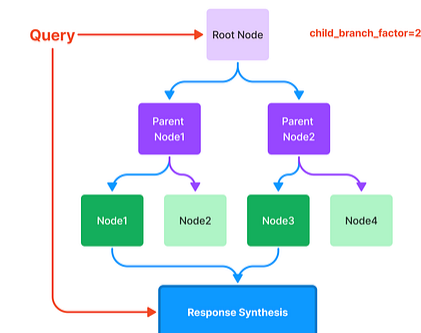
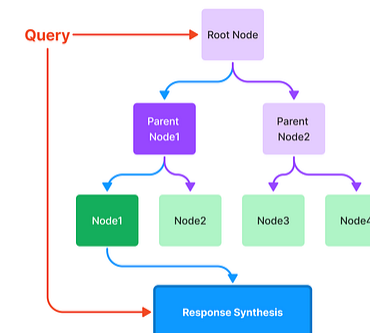
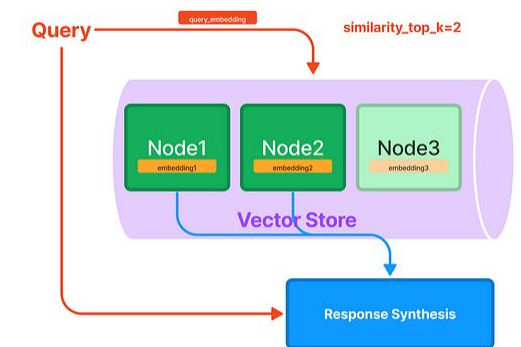
```
from llama_index import ServiceContext
service_context = ServiceContext.from_defaults(chunk_size=1000)
```

```
from llama_index import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader('data').load_data()
index = VectorStoreIndex.from_documents(documents, service_context=service_context)
query_engine = index.as_query_engine()
response = query_engine.query("What did the author do growing up?")
print(response)
```

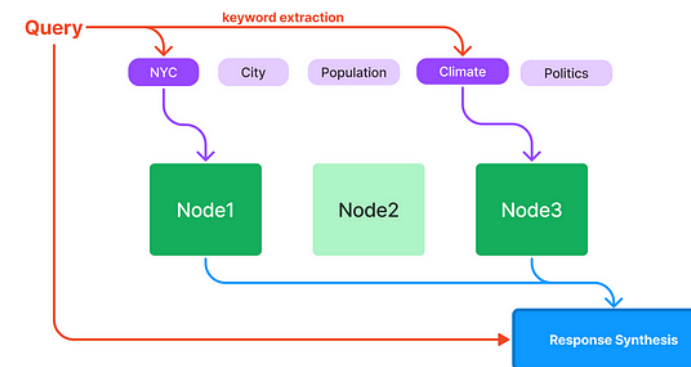
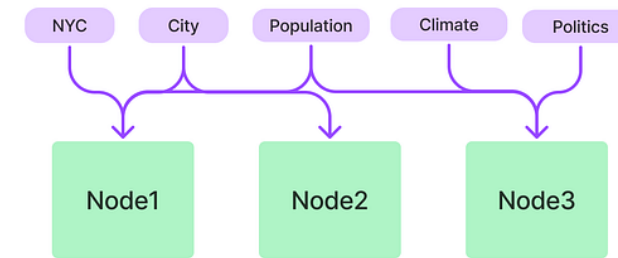
LLAMAINDEX

- Querying a vector store index involves fetching the top-k most similar Nodes, and passing those into Response Synthesis module.
- **The tree index** is a tree-structured index,
 - each node is a summary of the children's nodes.
 - During index construction, the tree is constructed in a bottoms-up fashion until we end up with a set of root nodes.
- The tree index builds a hierarchical tree from a set of Nodes (which become leaf nodes in this tree).
- Querying a tree index involves traversing from root nodes down to leaf nodes.



LLAMA INDEX

- The keyword table index extracts keywords from each Node and builds a mapping from each keyword to the corresponding Nodes of that keyword.
- During query time, LlamaIndex extracts relevant keywords from the query and match those with pre-extracted Node keywords to fetch the corresponding Nodes.
- The extracted Nodes are passed to our Response Synthesis module.



LLAMAINDEX

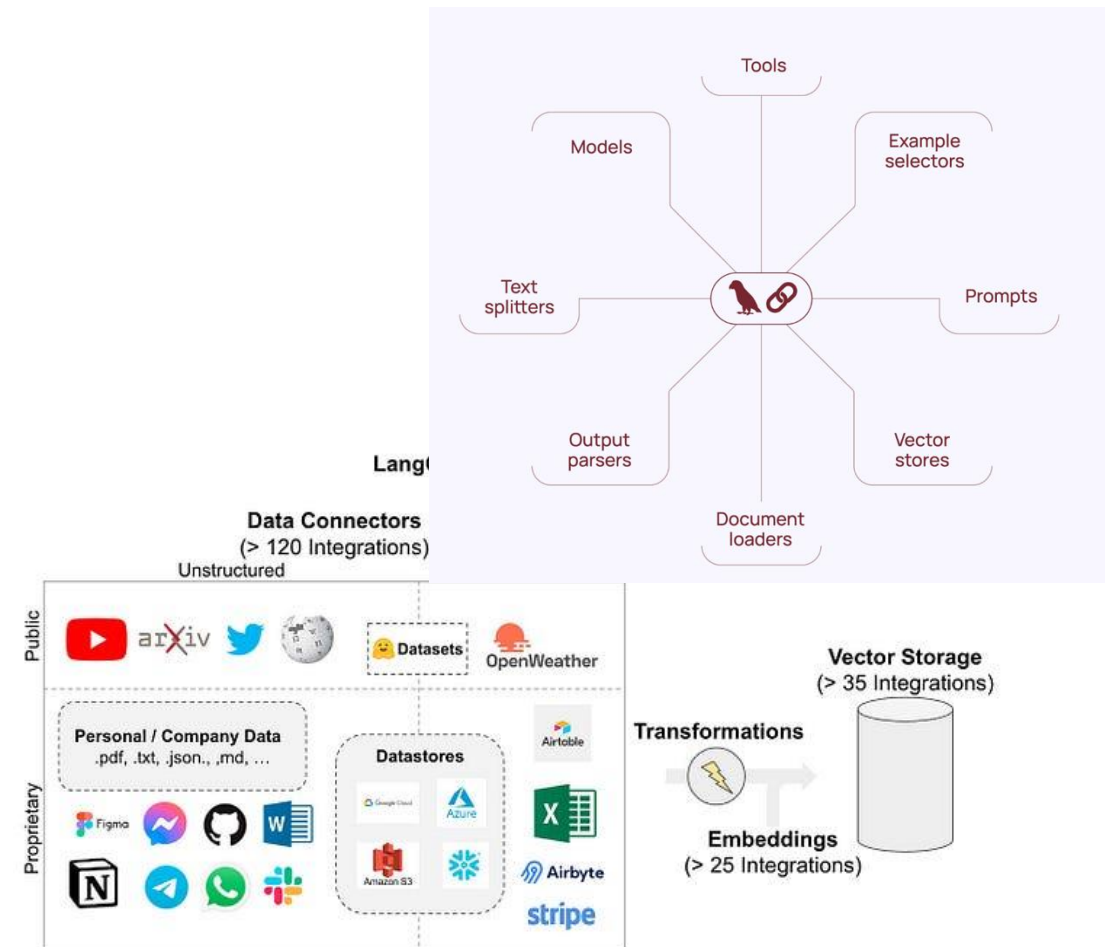
- LlamaIndex allows programmers to create composite indices by building indices on top of existing ones.
- For example indices at multiple levels,
 - lower-level indices for individual documents and
 - higher-level indices for groups of documents
- In this scenario
 - create a tree index for the text within each document.
 - generate a list index that covers all of the tree indices for your entire document collection.
- Current limitations
 - Text chunks do not have a complete global context,
 - There is a need for careful tuning of top-k / similarity score thresholds
 - Embeddings may not always select the most appropriate context for a question
 - The expense of indexing is a crucial factor to consider
 - The time of document indexing

LLAMA INDEX

Type	Purpose	LLM Call aka Embedding Fee
Vector Store Index	Very simple to use, allows answering a query over a large corpus of data	Yes
List Index	List index is useful for synthesizing an answer that combines information across multiple data sources	No, only when perform data retrieval
Tree Index	It is useful for summarizing a collection of documents	No, only when perform data retrieval
Keyword Table Index	It is useful for routing queries to the disparate data source	Yes
Composability Graph Index	It is useful for building a knowledge graph and stack multiple indices	Yes
Pandas Index and SQL Index	It is useful for structured data	No
Document Summary Index	Premade for answer-questioning purpose.	Yes

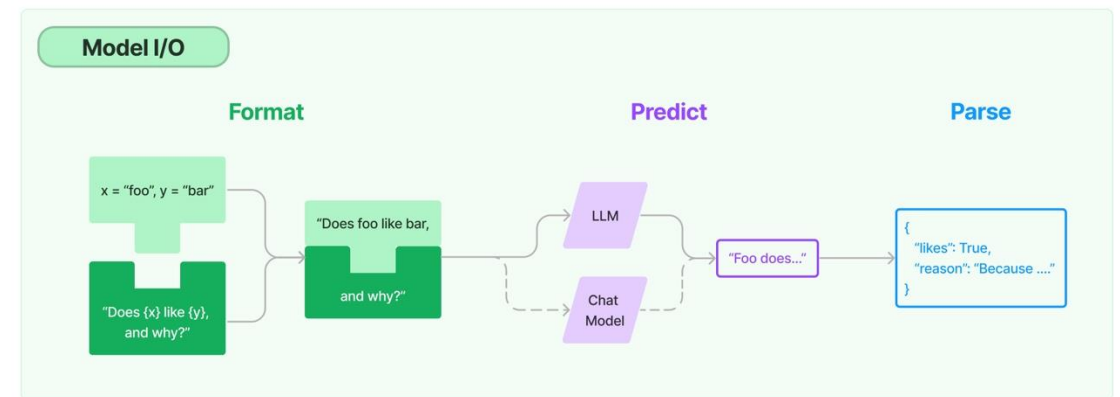
LANGCHAIN

- LangChain is a framework for developing applications powered by language models
 - Flowise is an open source Graphic User Interface to build LLM based applications on LangChain.
 - LangFlow for LangChain is a user-friendly graphical interface created to facilitate the testing & developing of applications and command sequences on LangChain.
- LangChain provides standard, extendable interfaces and external integrations for the following modules
 - Model I/O: Interface with language models
 - Retrieval: Interface with application-specific data
 - Chains: Construct sequences of calls
 - Agents: Let chains choose which tools to use given high-level directives
 - Memory: Persist application state between runs of a chain
 - Callbacks Log: and stream intermediate steps of any chain



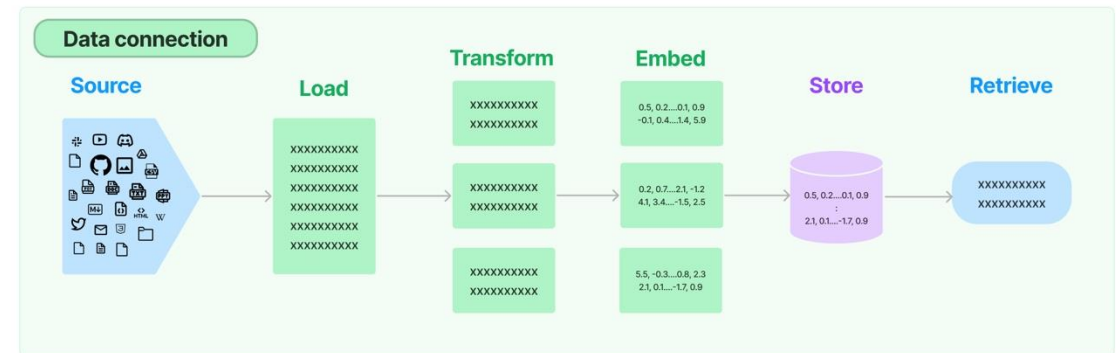
LANGCHAIN

- Model I/O
 - It is the core element of any language model application
 - Prompts: Templatize, dynamically select, and manage model inputs
 - Language models: Make calls to language models through common interfaces
 - Output parsers: Extract information from model outputs
 - Prompt templates are pre-defined recipes for generating prompts for language models.
- A prompt template may include instructions, few-shot examples, and specific context and questions appropriate for a given task.
 - LangChain provides tooling to create and work with prompt templates.
 - LangChain strives to create model agnostic templates to make it easy to reuse existing templates across different language models.



LANGCHAIN

- Retrieval. LangChain provides all the building blocks for RAG applications
- Document loaders Load documents from many different sources.
 - LangChain provides over 100 different document loaders
- Document transformers LangChain provides several different algorithms for doing this, as well as logic optimized for specific document types (code, markdown, etc).
- Text embedding models LangChain provides integrations with over 25 different embedding providers and methods, from open-source to proprietary API
- Vector stores LangChain provides integrations with over 50 different vectors tores
- Retrievers LangChain supports many different retrieval algorithms from simple semantic search to ensemble retriever



LANGCHAIN

- The indexing API allow you to load and keep in sync documents from any source into a vector store.
- When indexing content, hashes are computed for each document, and the following information is stored:
 - the document hash (hash of both page content and metadata)
 - write time
 - the source id

Cleanup Mode	De-Duplicates Content	Parallelizable	Cleans Up Deleted Source Docs	Cleans Up Mutations of Source Docs and/or Derived Docs	Clean Up Timing
None	✓	✓	✗	✗	-
Incremental	✓	✓	✗	✓	Continuously
Full	✓	✗	✓	✓	At end of indexing

LANGCHAIN

- LangChain provides two high-level frameworks for "chaining" components.
- LangChain Expression Language (LCEL) a declarative way to compose chains together.
 - The most visible part of LCEL is that it provides an intuitive and readable syntax for composition. But more importantly, it also provides first-class support for:
 - streaming, async calls, batching, parallelization, retries, fallbacks, tracing, ...

```
from operator import itemgetter

from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.schema import StrOutputParser

prompt1 = ChatPromptTemplate.from_template("what is the city {person} is from?")
prompt2 = ChatPromptTemplate.from_template(
    "what country is the city {city} in? respond in {language}"
)

model = ChatOpenAI()

chain1 = prompt1 | model | StrOutputParser()

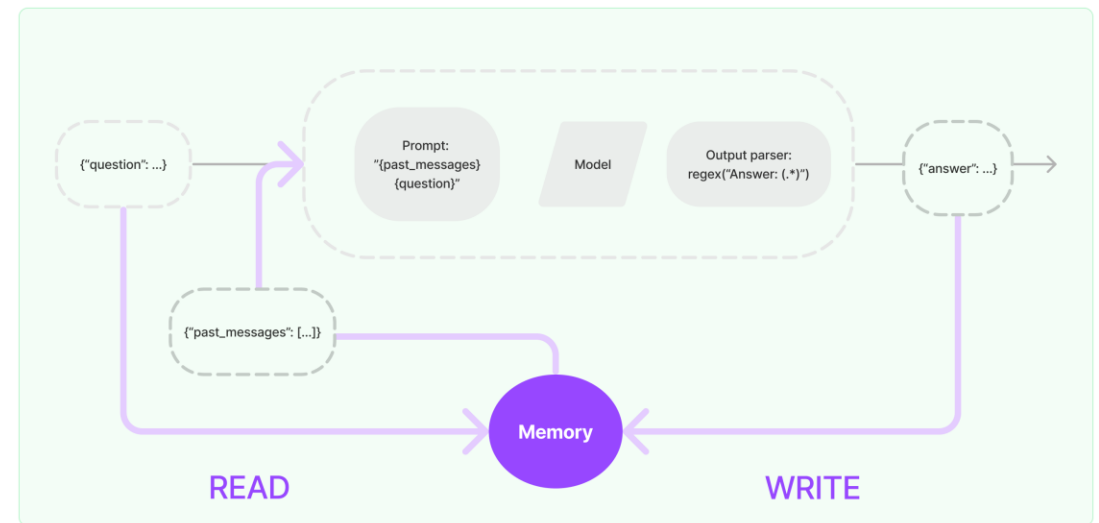
chain2 = (
    {"city": chain1, "language": itemgetter("language")}
    | prompt2
    | model
    | StrOutputParser()
)

chain2.invoke({"person": "obama", "language": "spanish"})
```

```
'El país donde se encuentra la ciudad de Honolulu, donde nació Barack Obama, el 44º Presidente de los Estado
```


LANGCHAIN

- Most LLM applications have a conversational interface.
 - An essential component of a conversation is being able to refer to information introduced earlier in the conversation.
- Memory component is the ability to store information about past interactions
- A memory system needs to support two basic actions: reading and writing. memory.
- A chain will interact with its memory system twice in a given run.
 - AFTER receiving the initial user inputs and BEFORE executing the core logic, a chain will READ from its memory system and augment the user inputs.
 - AFTER executing the core logic and BEFORE returning the answer, a chain will WRITE the inputs and outputs of the current run to memory

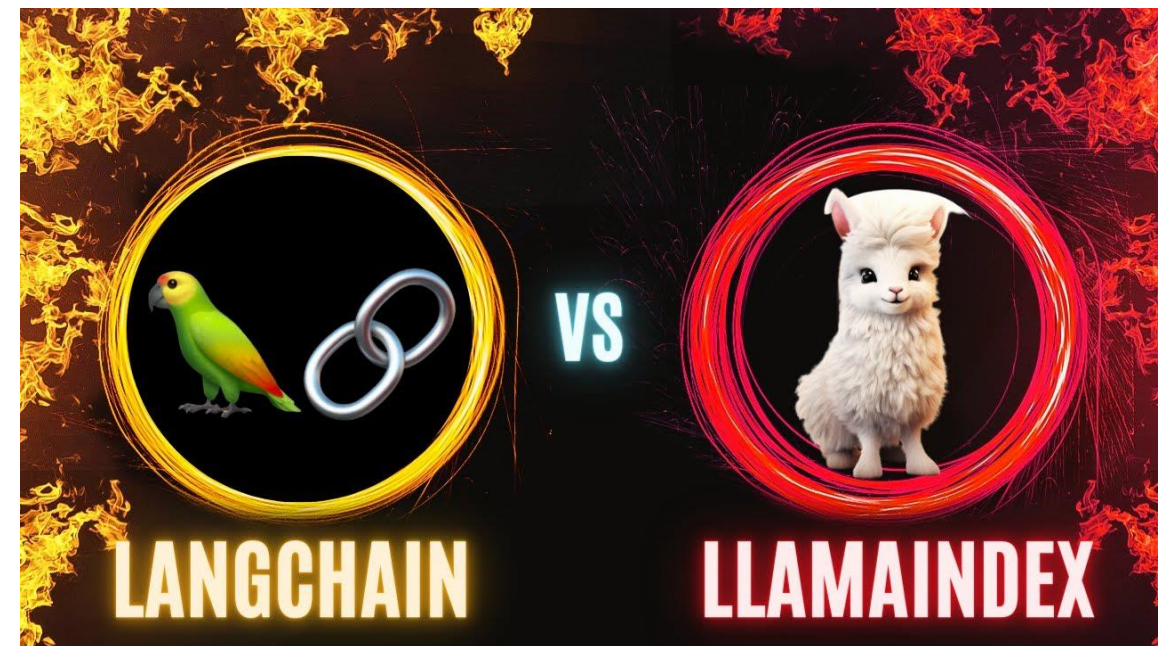


LANGCHAIN

- The core idea of agents is to use an LLM to choose a sequence of actions to take.
 - In chains, a sequence of actions is hardcoded (in code).
 - In agents, a language model is used as a reasoning engine to determine which actions to take and in which order.
- **AgentAction**: This is a dataclass that represents the action an agent should take.
- **AgentFinish**: This is a dataclass that signifies that the agent has finished and should return to the user.
- **intermediate_steps**: These represent previous agent actions and corresponding outputs that are passed around.
 - These are important to pass to future iteration so the agent knows what work it has already done.
 - This is typed as a `List[Tuple[AgentAction, Any]]`.
- LangChain provides a callbacks system that allows you to hook into the various stages of your LLM application.
- This is useful for logging, monitoring, streaming, and other tasks.

COMPARISON

- Tools offered by LLamaIndex:
 - Data Connectors, Data Indexes, Engines, Data Agents
- Tools offered by Langchain:
 - Model I/O, Retrieval, Chains, Memory, Agent, callback
- LLamaIndex focuses on bridging the gap between your data and LLMs
- LangChain offers a modular and extensible architecture that empowers developers to seamlessly combine LLMs with various data sources and services.





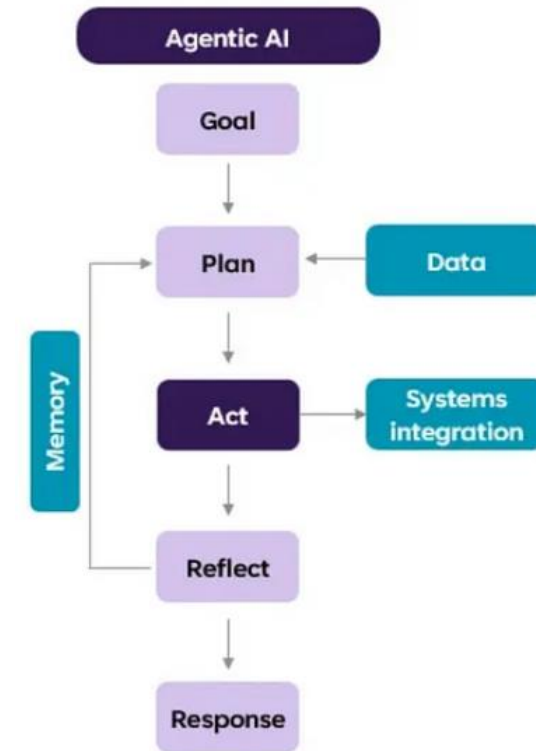
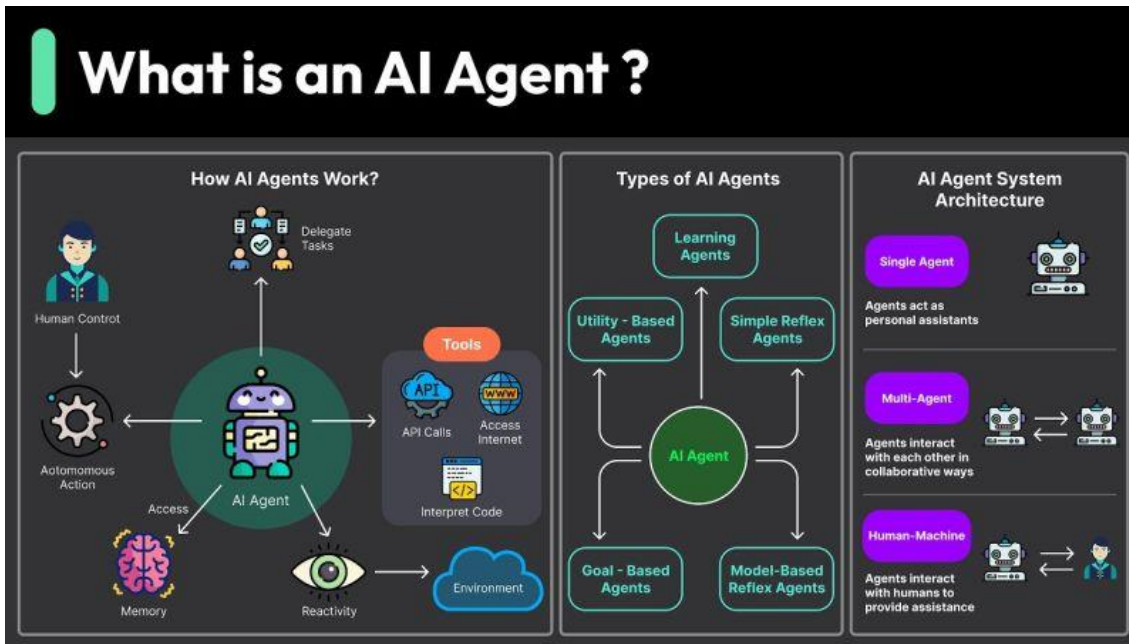
AGENTIC AI



AGENTIC AI

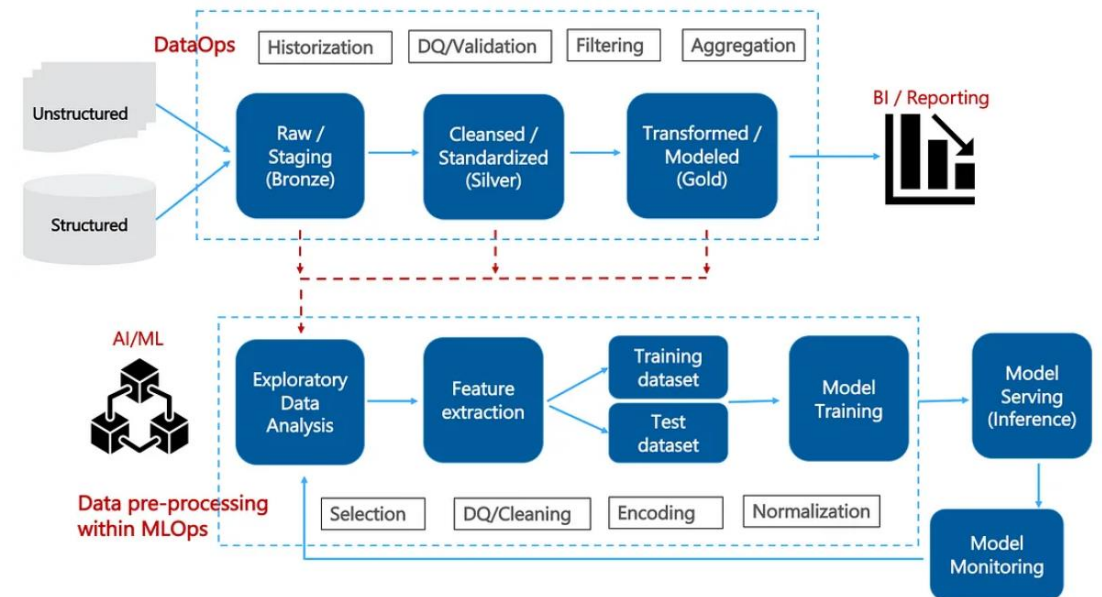
- AI agent as a theoretical concept existed since the 1950s as long as the field of artificial intelligence has
 - Earlier versions of AI or AI agents were rule-based systems that operated on a set of predefined rules to make decisions.
 - An AI agent is an autonomous software system that perceives its environment, makes decisions, and takes actions to achieve specific goals without constant human intervention
- **Autonomous:**
 - **Make decisions**
 - **Take Actions**
- When you look up a book you want to order on Amazon (goal)
 - you go through the list of books and prices, and make a decision on which book to buy
 - Change you with agent

AGENTIC AI



AGENTIC AI FOR DATA ENGINEER

- Source data, both structured and unstructured, is ingested into the Bronze layer
- where it is cleansed and standardized into the Silver layer
- with further modeling and transformation into the Gold layer.
- The recommended approach here is to complement the data processing strategy with directed acyclic graph (DAG) flows.
- Tools like Apache Airflow support the authoring, management and maintenance associated with DAG flows



AGENTIC AI FOR DATA ENGINEER

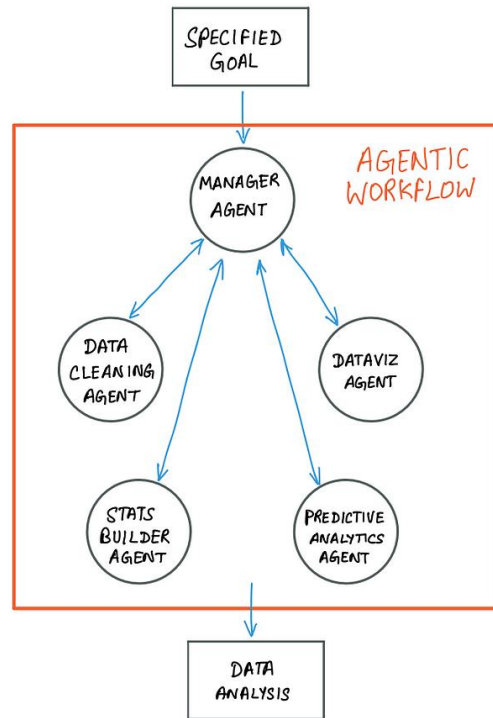
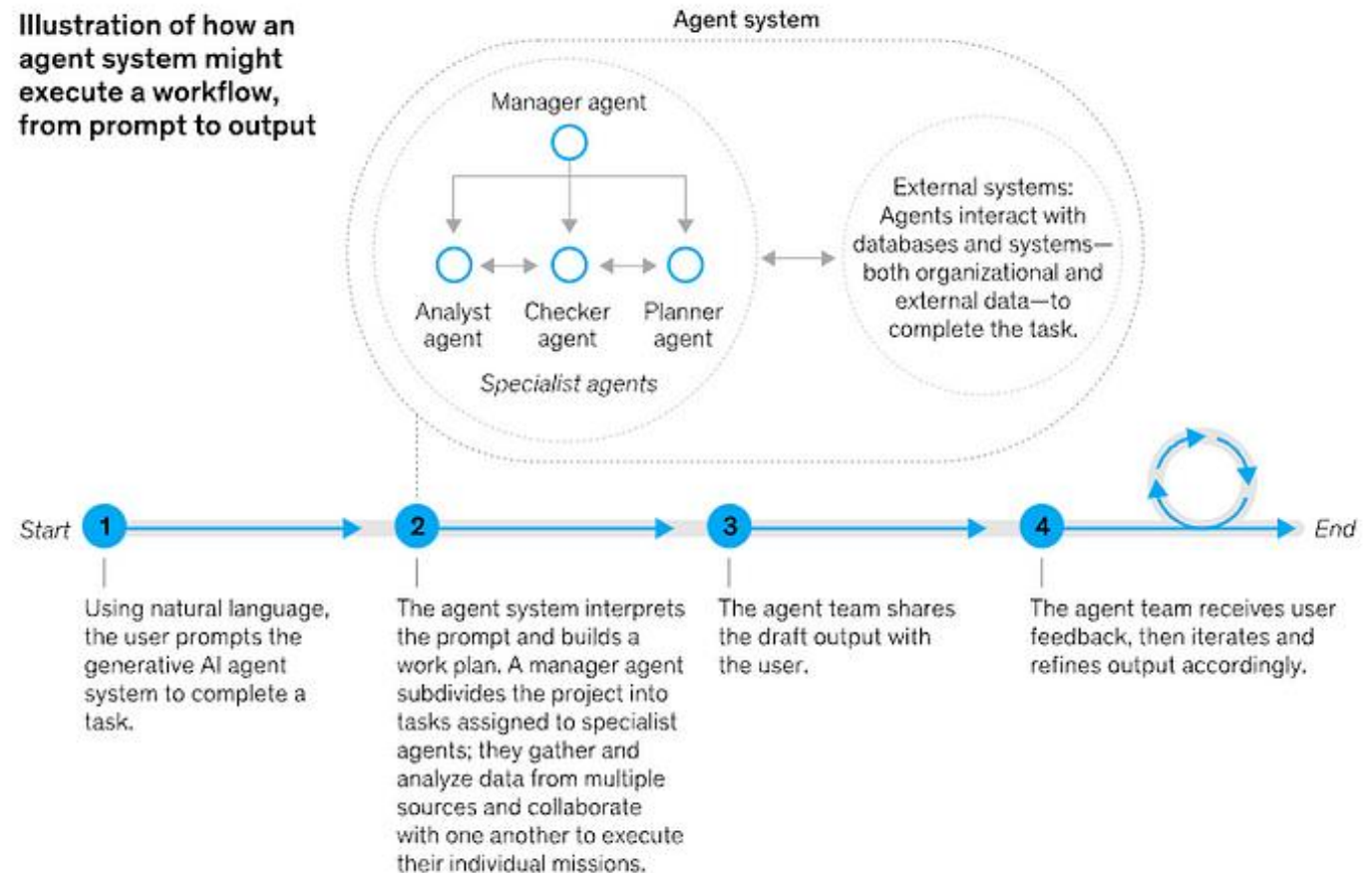
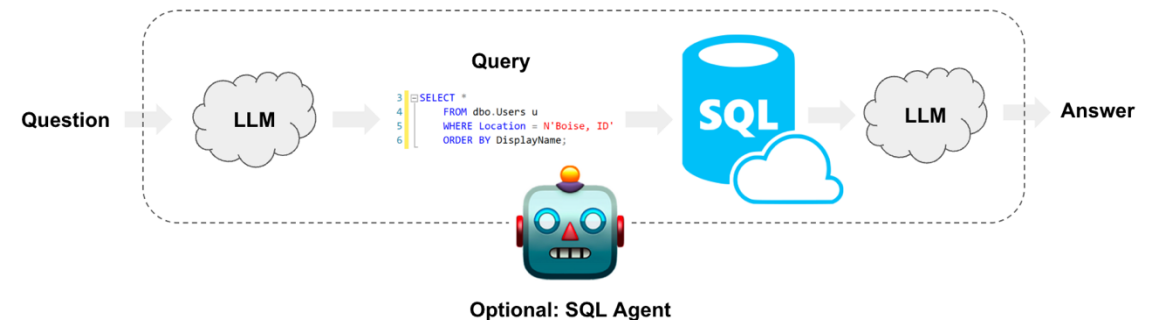


Illustration of how an agent system might execute a workflow, from prompt to output



A REAL EXAMPLE

- A system will allow us to ask a question about the data in a database and get back a natural language answer.
- At a high-level, the steps of these systems are:
 - **Convert question to SQL query:** Model converts user input to a SQL query.
 - **Execute SQL query:** Execute the query.
 - **Answer the question:** Model responds to user input using the query results.



A REAL EXAMPLE

- Agents leverage the reasoning capabilities of LLMs to make decisions during execution.
- Using agents allows you to offload additional discretion over the query generation and execution process.
- **Their behavior is less predictable than chains**

```
from langchain import hub

prompt_template = hub.pull("langchain-ai/sql-agent-system-prompt")
```

```
=====␣[1m System Message ␣[0m=====

You are an agent designed to interact with a SQL database.
Given an input question, create a syntactically correct ␣[33;1m␣[1;3m{dialect}␣[0m query to run, then look at t
Unless the user specifies a specific number of examples they wish to obtain, always limit your query to at most
You can order the results by a relevant column to return the most interesting examples in the database.
Never query for all the columns from a specific table, only ask for the relevant columns given the question.
You have access to tools for interacting with the database.
Only use the below tools. Only use the information returned by the below tools to construct your final answer.
You MUST double check your query before executing it. If you get an error while executing a query, rewrite the

DO NOT make any DML statements (INSERT, UPDATE, DELETE, DROP etc.) to the database.

To start you should ALWAYS look at the tables in the database to see what you can query.
Do NOT skip this step.
Then you should query the schema of the most relevant tables.
```

A REAL EXAMPLE

```
system_message = prompt_template.format(dialect="SQLite", top_
```

```
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent

agent_executor = create_react_agent(llm, tools, prompt=system
```

```
question = "Which country's customers spent the most?"
```

```
for step in agent_executor.stream(
    {"messages": [{"role": "user", "content": question}]},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

Run Metadata

HUMAN

Which country's customers spent the most?

AI

sql_db_list_tables call_YFrORGdbjAMJM1c92AgcPb6q

{}

YAML ↕

TOOL

sql_db_list_tables call_YFrORGdbjAMJM1c92AgcPb6q

1

"Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, Track"

JSON ↕

RAW 

AI

sql_db_schema call_CY7rItRNs3wwibZeCo6nEOk9

table_names: Customer

sql_db_schema call_Y1gEC0tYud3LaDFjhBswVvZw

table_names: Invoice

A REAL EXAMPLE

TOOL

sql_db_schema call_CY7rItRNs3wwibZe

```
CREATE TABLE Customer (
  "Country" NVARCHAR(40),
  "PostalCode" NVARCHAR(10),
  "Phone" NVARCHAR(24),
  "Fax" NVARCHAR(24),
  "Email" NVARCHAR(60) NOT NULL,
  "SupportRepId" INTEGER,
  PRIMARY KEY ("CustomerId"),
  FOREIGN KEY("SupportRepId") REFERENCES SupportRep (

```

/*

3 rows from Customer table:

CustomerId	FirstName	LastName	Country	PostalCode	Phone	Fax	Email	SupportRepId
1	Luis	Gonçalves	Embraer - Emp					
	José dos Campos	SP	Brazil	12227-000				
2	Leonie	Köhler	None	Theodor-H				
	None	leonekohler@surfeu.de		5				
3	François	Tremblay	None	14				
	None	ftremblay@gmail.com		3				

*/

YAML ↕

sql_db_schema call_Y1gECOtYud3LaDFjhBswVvZw

```
CREATE TABLE Invoice (
  "InvoiceId" INTEGER NOT NULL,
  "CustomerId" INTEGER NOT NULL,
  "InvoiceDate" DATETIME NOT NULL,
  "BillingAddress" NVARCHAR(70),
  "BillingCity" NVARCHAR(40),
  "BillingState" NVARCHAR(40),
  "BillingCountry" NVARCHAR(40),
  "BillingPostalCode" NVARCHAR(10),
  "Total" NUMERIC(10, 2) NOT NULL,
  PRIMARY KEY ("InvoiceId"),
  FOREIGN KEY("CustomerId") REFERENCES "Customer" ("CustomerId")
)
```

/*

3 rows from Invoice table:

InvoiceId	CustomerId	InvoiceDate	BillingAddress	BillingCity	BillingState	BillingCountry	BillingPostalCode	Total
1	2	2021-01-01 00:00:00	Theodor-Heuss-Straße 34	Stuttgart	None	Germany	70174	1.98
2	4	2021-01-02 00:00:00	Ullevålsveien 14	Oslo	None	Norway	0171	3.96
3	8	2021-01-03 00:00:00	Grétrystraat 63	Brussels	None	Belgium	1000	5.94

*/

YAML ↕

RAW 📄

A REAL EXAMPLE

AI

sql_db_query_checker `call_czLwzKhWyaQI2WaT10kANnX6`

```
query: |-
  SELECT c.Country, SUM(i.Total) AS TotalSpent
  FROM Customer c
  JOIN Invoice i ON c.CustomerId = i.CustomerId
  GROUP BY c.Country
  ORDER BY TotalSpent DESC
  LIMIT 5;
```

YAML ↕

TOOL

sql_db_query_checker `call_czLwzKhWyaQI2WaT10kANnX6`

```
|-
  ```sql
 SELECT c.Country, SUM(i.Total) AS TotalSpent
 FROM Customer c
 JOIN Invoice i ON c.CustomerId = i.CustomerId
 GROUP BY c.Country
 ORDER BY TotalSpent DESC
 LIMIT 5;
  ```
```

YAML ↕

AI

sql_db_query `call_Mwn5P6vMqLa9flqqNeRt3aEC`

```
query: |-
  SELECT c.Country, SUM(i.Total) AS TotalSpent
  FROM Customer c
  JOIN Invoice i ON c.CustomerId = i.CustomerId
  GROUP BY c.Country
  ORDER BY TotalSpent DESC
  LIMIT 5;
```

YAML ↕

TOOL

sql_db_query `call_Mwn5P6vMqLa9flqqNeRt3aEC`

```
"[('USA', 523.06), ('Canada', 303.96), ('France', 195.1), ('Brazil', 190.1), ('Germany', 156.48)]"
```

YAML ↕

RAW 

AI

The country whose customers spent the most is the **USA**, with a total spending of **\$523.06**. Here are the top five countries by customer spending:

1. **USA**: \$523.06
2. **Canada**: \$303.96
3. **France**: \$195.10
4. **Brazil**: \$190.10
5. **Germany**: \$156.48

RAW 

AGENTE DI VIAGGIO

```
tools
├── main.py
├── trip_agents.py
├── trip_tasks.py
├── .pythonlibs
├── poetry.lock
├── pyproject.toml
├── .replit
└── replit.nix

main.py
15 def run(self):
16     agents = TripAgents()
17     tasks = TripTasks()
18
19     city_selector_agent = agents.city_selection_agent
20     local_expert_agent = agents.local_expert()
21     travel_concierge_agent = agents.travel_concierge()
22
23     identify_task = tasks.identify_task(
24         city_selector_agent,
25         self.origin,
26         self.cities,
27         self.interests,
28         self.date_range
29     )
30     gather_task = tasks.gather_task(
31         local_expert_agent,
32         self.origin,
33         self.interests,
34         self.date_range
35     )
36     plan_task = tasks.plan_task(
37         travel_concierge_agent,
38         self.origin,
39         self.interests,
40         self.date_range
41     )
42
43     crew = Crew(
44         agents=[
45             city_selector_agent, local_expert_agent,
```

```
def identify_task(self, agent, origin, cities, interests, range):
    return Task(description=dedent(f"""
        Analyze and select the best city for the trip based
        on specific criteria such as weather patterns, seasonal
        events, and travel costs. This task involves comparing
        multiple cities, considering factors like current weather
        conditions, upcoming cultural or seasonal events, and
        overall travel expenses.

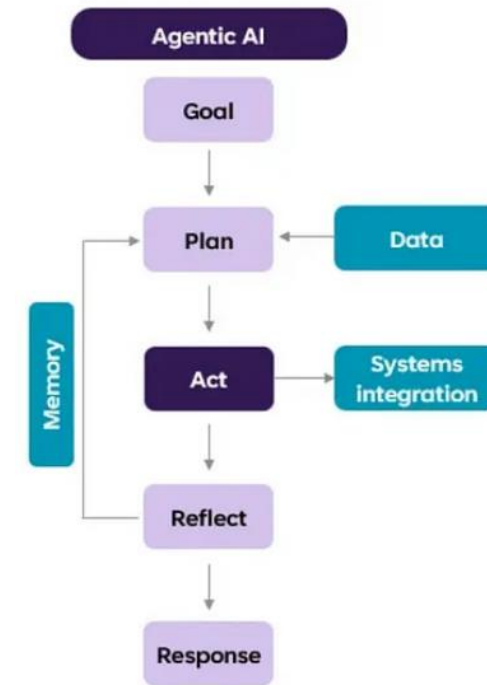
        Your final answer must be a detailed
        report on the chosen city, and everything you found out
        about it, including the actual flight costs, weather
        forecast and attractions.
        {self.__tip_section()}

        Traveling from: {origin}
        City Options: {cities}
        Trip Date: {range}
        Traveler Interests: {interests}
    """),
        agent=agent)

def gather_task(self, agent, origin, interests, range):
    return Task(description=dedent(f"""
        As a local expert on this city you must compile an
        in-depth guide for someone traveling there and wanting
        to have THE BEST trip ever!
        Gather information about key attractions, local customs,
        special events, and daily activity recommendations.
        Find the best spots to go to, the kind of place only a
        local would know.
    """),
        agent=agent)
```

HOW IT IS DONE AN AGEN

- An AI agent is composed of:
- **Objective:** receives a high-level task (e.g., “Write a sales report”).
- **Planner:** breaks the task down into logical steps and decides their sequence.
- **Action:** produces an output (report, code, email sending, etc.).
- **LLM:** generates text, code, or commands based on the context.
- **System integration:** can access external tools (APIs, databases, filesystem).
- **Memory:** keeps track of state, intermediate results, and context.
- **Reflect:** the agent evaluates whether the step taken is correct, whether it needs to be reformulated, or whether the strategy should be changed.
- **Response:** decides how to respond to the user or the environment, based on what it has done and reflected upon.



PLANNER

Aside from the Apple Remote, what other devices can control the program Apple Remote was originally designed to interact with?

- In the planning phase, the agent decides which steps must be carried out to achieve the results.
- **Workflow definition**
- There are two types of planners:
- **Implicit:** the LLM decides “on the fly” which steps to execute directly within the prompt.
- **Explicit:** the planner builds a plan before taking action.
- **Prompt chaining**
- Execution is carried out through **tool calling**.

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



MEMORY

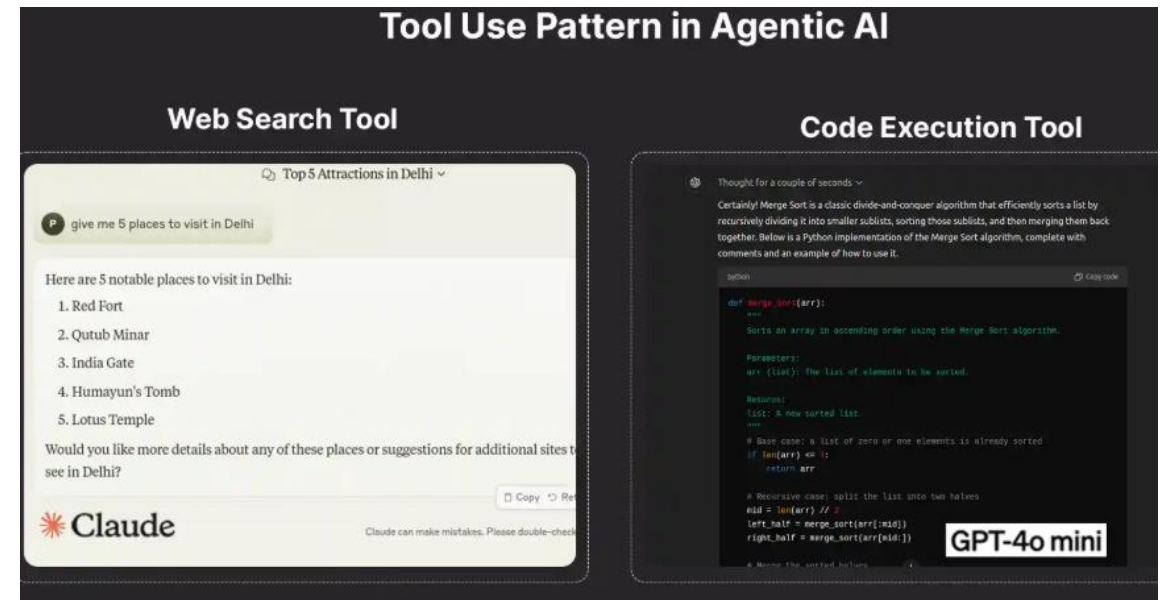
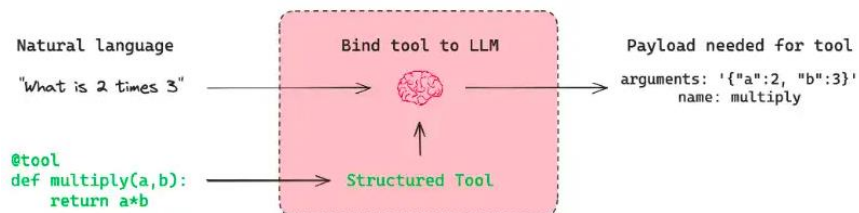
- Memory is a system that remembers something about previous interactions.
- Agents do not necessarily need memory systems.
- Simple reflex agents, for example, perceive real-time information about their environment and act on it or pass that information along.
- LLMs cannot remember things
 - except context window
 - short term memory
- The memory component must be added.
- Long-term memory (LTM) allows AI agents to store and recall information across different sessions, making them more personalized and intelligent over time.
- LTM is designed for permanent storage, often implemented using databases, knowledge graphs or vector embeddings.
- One of the most effective techniques for implementing LTM is retrieval augmented generation (RAG), where the agent fetches relevant information from a stored knowledge base to enhance its responses.

TOOL USE

- It is the capacity of interacting with external client-side tools and functions, allowing you to equip agent with your own custom tools to perform a wider variety of tasks.
 - Db
 - Web browser
 - Email
 - Python execution
 - Api...
- The AI relies on inferred invocation figuring out which tool to use based on what the user wants
 - While traditional software is built on explicit invocation, with a developer deciding every step in advance.
- AI can interact with real-world applications smoothly.
- Existing procedures cannot simply be turned into tools.
 - Procedures are designed for structured code execution,
 - Tools must be designed for AI agents that interpret natural language, infer intent, and make autonomous decisions.
 - This difference means that an additional layer must be built

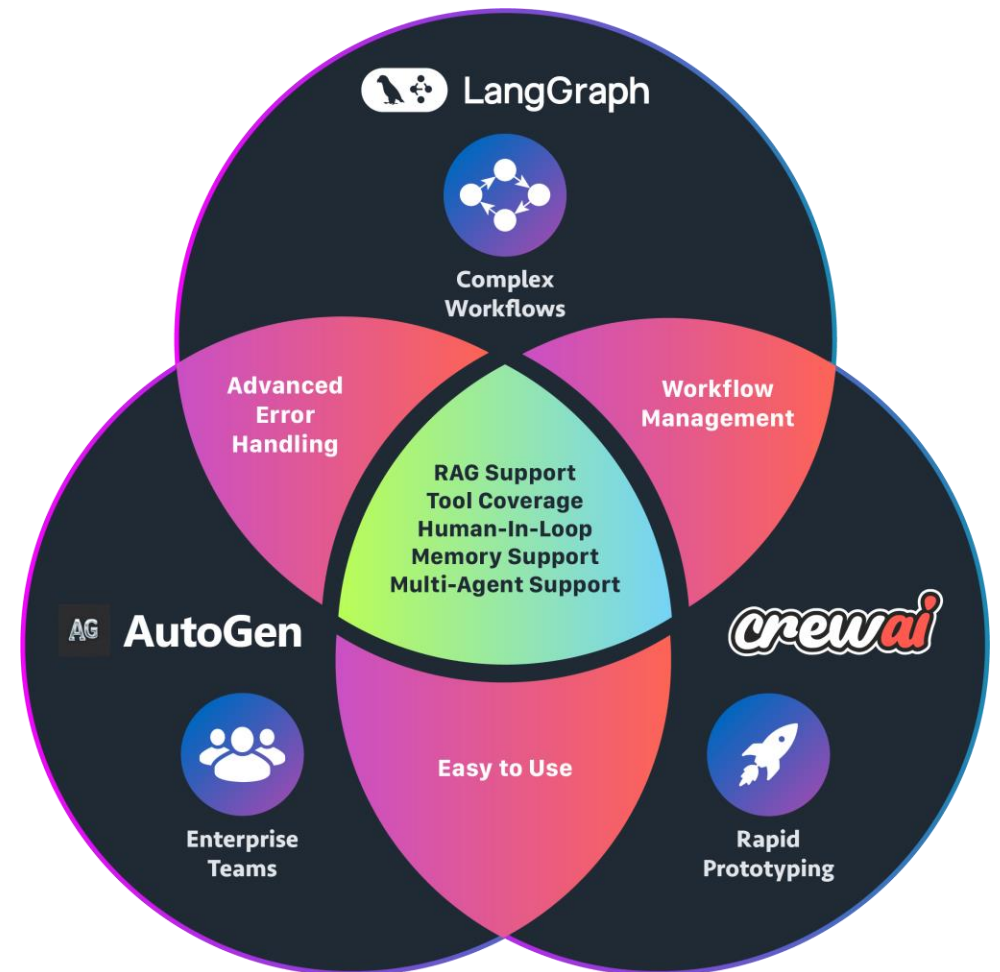
TOOL USE

1. **Input:** User asks “What is 2 times 3?”
2. **Interpretation:** The LLM recognizes this as a mathematical operation.
3. **Tool Selection:** The LLM selects the multiply tool.
4. **Payload Creation:** It extracts relevant arguments (a: 2 and b: 3), prepares a payload, and calls the tool.
5. **Execution:** The tool performs the operation ($2 * 3 = 6$), and the result is passed back to the LLM to present to the user.



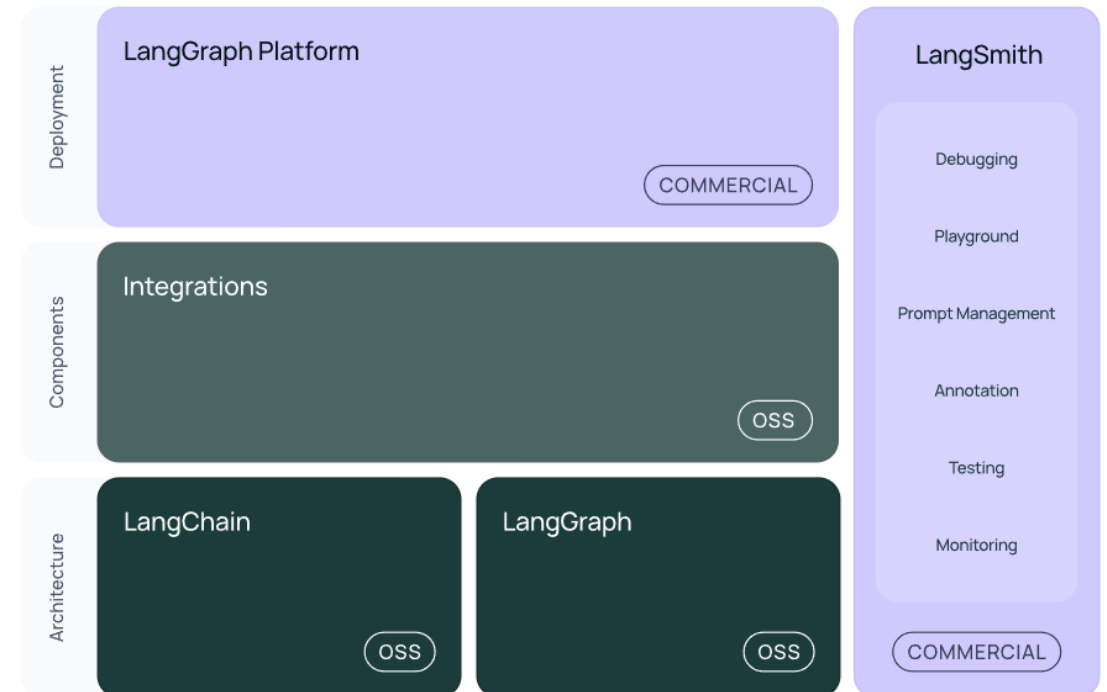
TOOL

- LangGraph
- AutoGen (Microsoft)
- CrewAI



LANGGRAPH

- Un framework Python per costruire agenti LLM con tool, memory e planner.
- Molto usato per creare chatbot, assistenti, e flussi agentici con strumenti esterni
- Caratteristiche principali
 - Supporta tool use, memory, planner, retriever
- Estendibile con moduli personalizzati
- Funziona con OpenAI, Anthropic, Cohere, HuggingFace...



LANGGRAPH

```
# Import relevant functionality
from langchain_anthropic import ChatAnthropic
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import create_react_agent

# Create the agent
memory = MemorySaver()
model = ChatAnthropic(model_name="claude-3-sonnet-20240229")
search = TavilySearchResults(max_results=2)
tools = [search]
agent_executor = create_react_agent(model, tools, checkpoint=memory)
```

```
# Use the agent
config = {"configurable": {"thread_id": "abc123"}}
for step in agent_executor.stream(
    {"messages": [HumanMessage(content="hi im bob! and i live in sf")]},
    config,
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

```
for step in agent_executor.stream(
    {"messages": [HumanMessage(content="whats the weather where I live?")]},
    config,
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

```
=====␣[1m Human Message ␣[0m=====

whats the weather where I live?
=====␣[1m Ai Message ␣[0m=====

[{'text': 'To get the current weather for your location in San Francisco, I can use the tavily_search_results_json
Tool Calls:
  tavily_search_results_json (toolu_01AKa2MErG1CU3zRiGsvpBud)
Call ID: toolu_01AKa2MErG1CU3zRiGsvpBud
Args:
  query: san francisco weather
=====␣[1m Tool Message ␣[0m=====
Name: tavily_search_results_json

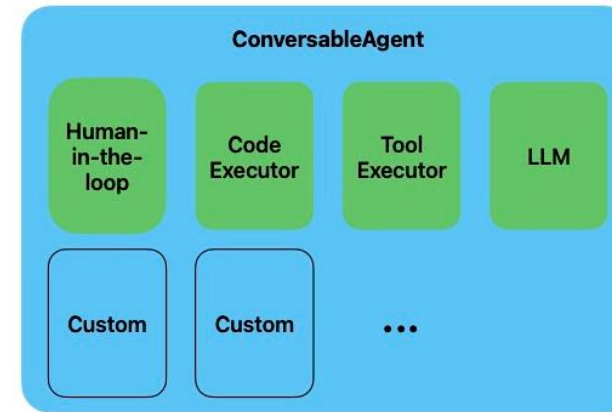
[{"url": "https://www.weatherapi.com/", "content": "'location': {'name': 'San Francisco', 'region': 'California'
=====␣[1m Ai Message ␣[0m=====

The search results provide the current weather conditions and forecast for San Francisco. According to the data

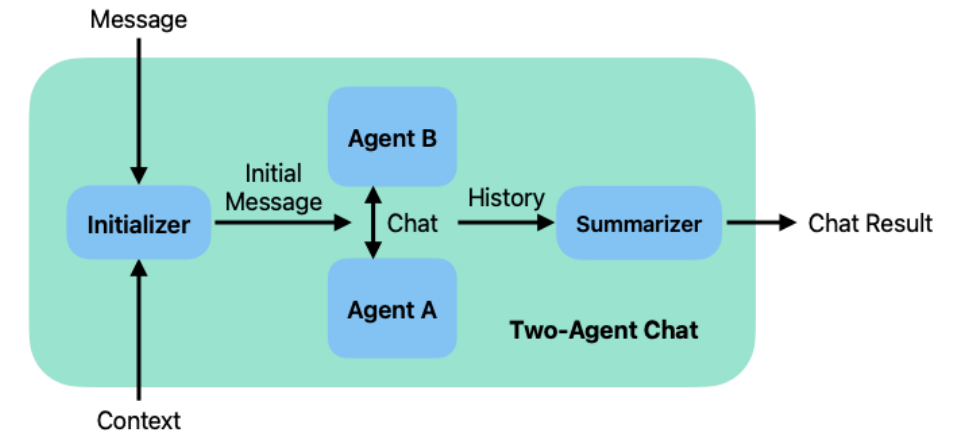
So in summary, it's a cool, rainy day currently in San Francisco where you live, Bob. Let me know if you need an
```

AUTOGEN

- Framework open-source di Microsoft per costruire sistemi multi-agente conversazionali
- Gli agenti parlano tra loro per risolvere task complessi
- Ogni agente ha: ruolo, profilo, funzione, toolset
- Supporta: reflection, planning, code execution, API chaining
- Utile per più agenti che collaborano
 - Coding, task iterativi..



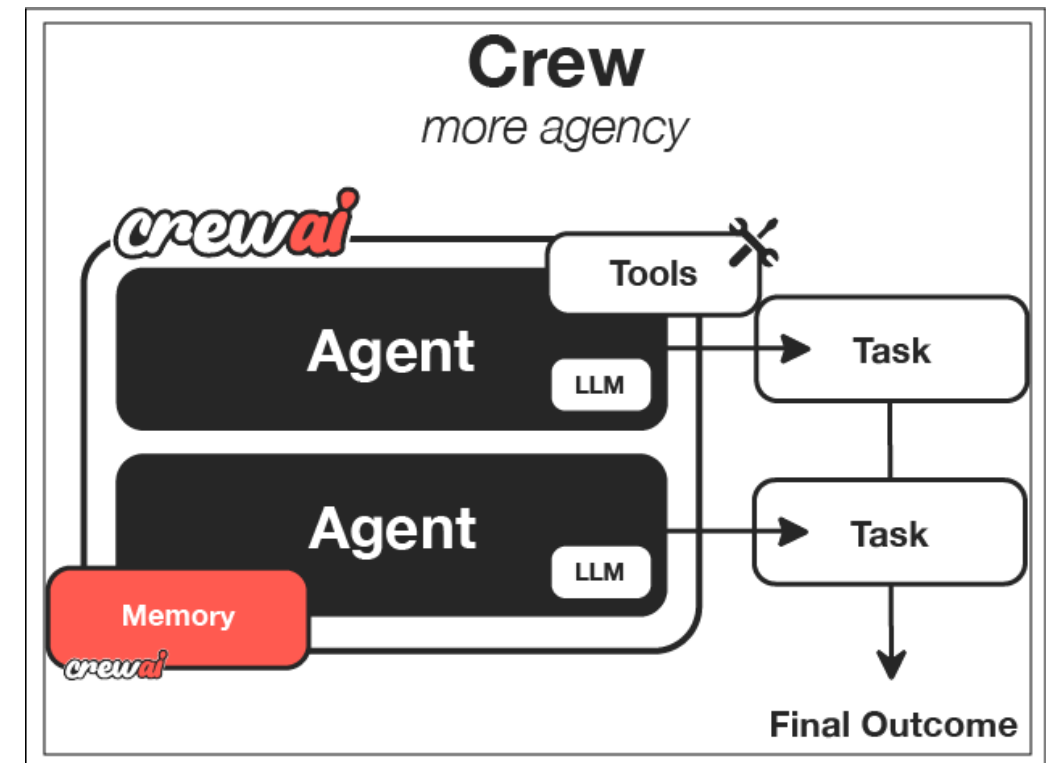
E.g., "What is triangle inequality?"



E.g., summary_method: reflection_with_llm

CREWAI

- Un framework per creare team di agenti con ruoli specifici e task assegnati
- Ogni agente può specializzarsi:
 - Ricercatore, Scrittore, coder
- Agenti con ruolo + tool + descrizione
- Supporta task sequenziali o paralleli
- Facile da usare e ben documentati
- Utile per costruire team virtuali di agenti
- Ideale per reporting, analisi, scrittura collaborativa



CREWAI

| Component | Description | Key Features |
|-----------|----------------------------|--|
| Crew | The top-level organization | <ul style="list-style-type: none">• Manages AI agent teams• Oversees workflows• Ensures collaboration• Delivers outcomes |
| AI Agents | Specialized team members | <ul style="list-style-type: none">• Have specific roles (researcher, writer)• Use designated tools• Can delegate tasks• Make autonomous decisions |
| Process | Workflow management system | <ul style="list-style-type: none">• Defines collaboration patterns• Controls task assignments• Manages interactions• Ensures efficient execution |
| Tasks | Individual assignments | <ul style="list-style-type: none">• Have clear objectives• Use specific tools• Feed into larger process• Produce actionable results |

- The Crew organizes the overall operation
- AI Agents work on their specialized tasks
- The Process ensures smooth collaboration
- Tasks get completed to achieve the goal